Fractal Intelligent Memory (FIM-32): Deterministic Resolution of Stochastic Heaps via Lattice-Bound Search Oracles

Andrew Jonathan Fine

May 31, 2026

Abstract

This paper introduces Fractal Intelligent Memory (FIM-32), a novel computer memory architecture designed to eliminate the latency variance and unpredictability inherent in traditional stochastic heap management. By organizing system memory as a rigid, deterministic multi-tiered lattice structured into geometric Sectors, Rows, and Planes, FIM-32 treats structural allocation as a constant-time coordinate address resolution task. We eliminate search-and-sort execution loops by routing access paths through discrete, boundary-enforced hardware search oracles. Our formal analysis demonstrates that matching allocation state metrics to a strict spatial occupancy limit (Γ) provides bounded path termination and absolute execution predictability. We validate this approach through concrete Dyalog APL tracking loop implementations, establishing that FIM-32 satisfies the rigorous timing constraints required for predictable, high-performance deterministic computing systems.

Contents

List of APL Source Listings **1**

1 Welcome to Iscandar. **1**
1.1 Batteries Are Included! . . . 1
1.1.1 The Welcome Example. . . . 1

2 Architectural Axioms **1**
2.1 Axiom I: The State Map (L) and Structural Arity ($\mathcal{A}$) . . . 1
2.1.1 The Cube Namespace . . . 1
2.2 Axiom II: Hierarchical Occupancy Counters (Γ) . . . 1
2.3 Axiom III: Enforcing an Occupancy Limit (Φ) . . . 2
2.4 Axiom IV: The Contiguity Guarantee and Spatial Entropy . . . 2
2.5 Axiom V: The Coordinate-Address Identity . . . 2

3 Formal Proof of Deterministic Path Termination **2**
3.1 The Steering-Based Complexity Model . . . 2
3.2 Guaranteed Path Termination . . . 3
3.3 Conclusion of Determinism . . . 3

4 Deterministic Coalescence and the Occupancy Limit **3**
4.1 Symmetric Coalescence Mechanics . . . 3
4.2 Geometric Feasibility and the Resolution Operator (Ω) . . . 3
4.3 Proof of Coalescence Invariant . . . 4

5 Observations **4**
5.1 As an Extension to Binary Buddy Systems . . . 4
5.2 As a Symmetric Voronoi Partition . . . 4
5.3 As Linear Programming . . . 5
5.4 Amortized Performance: Constant Time . . . 5
5.5 Worst Case Performance: Sub-Linear Time . . . 5
5.6 As Supporting Six Sigma Design . . . 5
5.7 Scalability via Depth Extension . . . 5
5.8 As Supporting Parallel Execution . . . 5

6 Conclusion **6**

7 Glossary of Terms **6**

8 Updating Memory State Spaces by Mating the Oracle Tables **6**

9 Description of Instruction Groups **12**
9.1 Phase I: The Admissibility Gate (Level 3) . . . 12
9.2 Phase II: The Plane-Level Resolution (Level 2) . . . 13
9.3 Phase III: The Row-Level Weld (Level 1) . . . 13
9.4 Phase IV: The Bit-Lattice Isolation (Level 0) . . . 13

10 Desk Check Summary: Nano and Tiny Resolutions ($L \leq \mathcal{A}$) **14**

11 Volumetric Analysis **14**
11.1 Methodology . . . 15
11.2 Test Data (generated from Section 10) . . . 15
11.3 Graphical Verification . . . 15
11.4 Buddy System Comparison Simulation . . . 15

A Bootstrap Mechanism (see Section 1, Batteries Included!) **19**

B Project Checklist. **19**

C The Bottle Opener: Use after Bootstrap. **19**
C.1 Project Directories 19

D Oracle Manufacture **20**
D.1 Center Oracle 21
D.2 Left Oracle 22
D.3 Right Oracle 22

E Dyalog APL Laboratory Listings. **23**
E.1 APL Definitions 23

1 Welcome to Iscandar.

Unlike allocation models using variable-time search algorithms [1], this architecture employs a hierarchical occupancy map to resolve coordinates deterministically. Extending the traditional binary buddy system to an arbitrary radix replaces it with a geometric steering mechanism.

1.1 Batteries Are Included!

The .tex source for this manuscript functions as a self-organizing research example. It can be processed by a PowerShell script to extracts Dyalog APL source listings directly from surrounding portions of this paper which document them best, to create a local research environment. This ensures the researcher does not need to manually configure local paths or binary repositories. See Appendix A and C for putting together the environment and Section 10 for its use. **Welcome.apl** below is the basic demonstration.

1.1.1 The Welcome Example.

See Listing 8 for `Welcome.apl`

2 Architectural Axioms

These axioms establish the geometric and mathematical constraints of the lattice. They transform each memory request into a coordinate resolution task, avoiding search-and-sort operations to maintain a fixed sequence of instructions.

2.1 Axiom I: The State Map (L) and Structural Arity ($\mathcal{A}$)

For modeling and testing purposes this paper demonstrates an k-level lattice L of fixed depth ($n = 3$) **defining** a total memory space. The architecture **maintains** a constant Arity ($\mathcal{A}$) of 32. A hierarchical progression of tiers **organizes** the lattice: the atomic Sector (S_0), the Row of Sectors (S_1), the Plane of Rows (S_2), and the Cube of Planes (S_3). Lattice coordinates (p, r, s) **determine** the address of any specific leaf node, where $p, r, s \in \{0, \ldots, \mathcal{A} - 1\}$. The geometry **maintains** invariance regardless of the physical byte-width the sector **occupies**.[2]

2.1.1 The Cube Namespace

This listing defines the memory allocator data structure, Iscandar's answer to the buddy system heap, in accordance with Axiom I.

2.2 Axiom II: Hierarchical Occupancy Counters (Γ)

Discrete occupancy counts (Γ) at every tier enable constant-time steering. Following bit-block representation principles, these counters generate a real-time density map of the lattice:

$$\text{Level 1 (Row):} \quad \Gamma_1 = \sum_{i=0}^{\mathcal{A}-1} \Gamma_{0,i} \tag{1}$$

$$\text{Level 2 (Plane):} \quad \Gamma_2 = \sum_{j=0}^{\mathcal{A}-1} \Gamma_{1,j} \tag{2}$$

$$\text{Level 3 (Cube):} \quad \Gamma_3 = \sum_{k=0}^{\mathcal{A}-1} \Gamma_{2,k} \tag{3}$$

[2] The Large allocator functions as a cross-tier searcher for sequence resolution, managing the fractal logic across the Plane manifold to satistfy multi-Row requests.

This hierarchical summation lets a Global Registrar at the Cube level execute an admissibility check in a single instruction cycle, imposing a fast-fail condition.

2.3 Axiom III: Enforcing an Occupancy Limit (Φ)

The **Iscandar** system **enforces** a mandatory occupancy boundary, the Occupancy Limit (Φ). Traditional allocators search for gaps as memory nears capacity; in contrast, this system **triggers** a deterministic refusal once Γ_3 reaches the Φ threshold based on the present occupancy request.

2.4 Axiom IV: The Contiguity Guarantee and Spatial Entropy

Any resolved coordinate sequence **represents** a continuous physical block in the **Iscandar** lattice. However, functional contiguity does not imply spatial perfection; a perfect Oracle would preclude fragmentation entirely. Our Oracles (Center, Left, and Right) prioritize determinism over maximum packing density.

2.5 Axiom V: The Coordinate-Address Identity

The architecture used here rejects the traditional separation of logical address space and physical memory geometry, by establishing a **Bijective Identity** between a machine-word pointer P (wrapped pointer, see section 5) and its corresponding lattice coordinate $\vec{C}$.

In a system of depth k and arity A, any specific sector is represented by a coordinate vector:

$$\vec{C} = (i_1, i_2, \ldots, i_k) \quad \text{where} \quad 0 \leq i_n < A \tag{4}$$

The address P is not an arbitrary token, but the scalar projection of this vector into the linear address space. The isomorphism is defined by the construction:

$$P = \sum_{n=1}^{k} i_n \cdot A^{k-n} \tag{5}$$

For the demonstration project of this paper, $(A = 32, k = 3)$, the address of a sector located at (Plane p, Row r, Sector s) is clearly enumerable from the geometry:

$$P = (p \cdot 32^2) + (r \cdot 32^1) + (s \cdot 32^0) \tag{6}$$

3 Formal Proof of Deterministic Path Termination

Theorem: A fixed-depth $\mathcal{L}$ of Arity $\mathcal{A} = 32$ **establishes** deterministic resolution by mapping every request to a finite execution path.

3.1 The Steering-Based Complexity Model

The resolution time T in the **Iscandar** architecture is a discrete function of the sub-lattice depth and the specific heuristic path $\mathcal{P}$ identified by the memory request size in sectors (L). Unlike systems where $T = O(\log N)$ or $O(N)$ based on heap fragmentation, the path selection in Iscandar uses a decision table to ensure each heuristic terminates.

$$T(\mathcal{L}) = f(\Gamma) \in \{\text{Path}_{\text{Nano}}, \text{Path}_{\text{Tiny}}, \text{Path}_{\text{Small}}, \text{Path}_{\text{Large}}, \text{Path}_{\text{Huge}}, \text{Path}_{\text{Cube}}\} \tag{7}$$

3.2 Guaranteed Path Termination

Every resolution path is constructed from a fixed sequence of *Instruction Groups* that eliminate runtime searching.

1. **Heuristic Selection:** The Global Registrar and intermediate sentinels use bitmask comparison to select the viable path.
2. **Atomic Resolution:** For leaf-level requests, the system executes either a single-pass (Nano) or a bimodal "Two-Bite" probe (Tiny or Small).
3. **Path Invariance:** Once a path is engaged, the algorithm terminates when the physical bit-offset is returned.

3.3 Conclusion of Determinism

Determinism in Iscandar is defined as the **guarantee of termination** within a bounded execution window for any given heuristic path, trading away spatial capacity for these predetermined paths.

4 Deterministic Coalescence and the Occupancy Limit

Maintaining temporal determinism in a dynamic environment **requires** a deallocation process (coalescence) **mirroring** the efficiency of the allocation logic. Operational stability of the **Iscandar** lattice uses discrete enforcement of the Occupancy Limit(Φ), **precluding** the high-entropy states typically associated with conventional allocators [9].

4.1 Symmetric Coalescence Mechanics

To maintain determinism our deallocation process mirrors the efficiency of the allocation logic, a state update performed upon the occupancy counters. The bit-address of any Sector (S_0) is a fixed function of its coordinates. Updating the occupancy counters (Γ) thus consumes a predictable instruction count (ι_{base}) identical to the initial resolution.

1. The atomic leaf node state Γ_0 is toggled to 0 (Available).
2. The change propagates immediately to the parent Row counter (Γ_1).
3. The Level-2 (Plane) and Level-3 (Cube) counters are updated through integer subtraction.

Because the lattice depth is fixed at $n = 3$, the time complexity for deallocation is defined by the same path-specific constraints as allocation.

4.2 Geometric Feasibility and the Resolution Operator (Ω)

Unlike traditional allocators that suffer from $O(N)$ search patterns as utilization approaches 100% [**?**], the Iscandar lattice treats the heap as a deterministic volume. The system does not utilize a static occupancy limit; instead, it enforces a dynamic gate based on the current requested length (L) and the total surviving capacity of the cube (Γ_3).

The Resolution Operator (Ω) acts as a volumetric circuit breaker. By evaluating a request against remaining free sectors before invoking any Oracle, we ensure resolution only proceeds within the feasible region of the memory manifold [**?**]. The decision logic is defined as:

$$\Omega(L) = \begin{cases} \text{Resolve}(p, r, s) & \text{if } L \leq (\Gamma_3 - \text{Occupancy}) \\ \text{Refuse} & \text{if } L > (\Gamma_3 - \text{Occupancy}) \end{cases} \tag{8}$$

4.3 Proof of Coalescence Invariant

Let Γ_{total} be the sum of all leaf nodes $S_{0,i}$. At any discrete time t, the hierarchical counters **must** satisfy the invariant:

$$\Gamma_{\text{Apex}} \text{ (a.k.a. } \Gamma_3) = \sum \Gamma_2 = \sum \Gamma_1 = \sum \Gamma_0 \tag{9}$$

Since deallocation is defined as:

$$\Gamma_x(t+1) = \Gamma_x(t) - 1 \quad \text{for all levels } x \in \{0, 1, 2, 3\} \tag{10}$$

along the specific resolution path, the invariant is preserved. Because this path is fixed at depth n, the lattice is restored to a consistent, deterministic state in nI_{base} time.

5 Observations

Earlier sections established **Iscandar** as deterministic within the primary scope of this paper. However, it exhibits properties warranting distinct architectural observations.

5.1 As an Extension to Binary Buddy Systems

The binary buddy system, pioneered by Knowlton [12] and Knuth [13], sought to minimize search latency through a power-of-two partitioning scheme ($R = 2$). While this approach facilitates efficient coalescence, its search complexity remains fundamentally constrained to $O(\log_2 n)$. As modern memory address spaces scale alongside processor word sizes, the increasing logarithmic depth of these trees drives significant latency. Furthermore, at high utilization, binary buddy systems trigger "stalling" behaviors; the search algorithm must traverse increasingly fragmented leaf nodes, which inevitably erodes deterministic performance.

We generalize buddy system principles to an **Arbitrary Radix** ($R = \mathcal{A}$). By shifting from a binary tree to a high-radix fractal lattice, we collapse the depth of the search space to a fixed architectural constant n. This structural shift moves memory management from the domain of probabilistic searching to the domain of **Geometric Coordinate Resolution**.

5.2 As a Symmetric Voronoi Partition

As detailed in the Architectural Axioms (Section 2.1), this lattice functions as a **Symmetric Voronoi Partition**, where the adjacency of "buddies" is a pre-baked geometric property of the address space rather than a calculated proximity, as shown in Section 5.2.

In this model, adjacency is defined by a partition's total volume **associating** with its handle (a wrapped pointer resolved in Section 9.4) rather than the Euclidean distance of individual points from a centroid. rather than the Euclidean distance of individual points from a centroid, as established in the geometric rules of the address space.

In a standard Voronoi partition, boundaries are dynamically computed as a function of the Euclidean distance to a specific "seed" or generator point. In the Iscandar lattice, while the "seed" is represented by the wrapped pointer, the partition boundaries are not computed at runtime, as defined by the invariant geometric properties of the lattice established in the Observations section. Instead, they are architecturally invariant, defined strictly by the Structural Arity ($\mathcal{A}$) and the hierarchical depth (k) established in the Axioms. This replaces the need for distance-based calculations with a pre-baked, static coordinate resolution:

$$\forall L \in \mathcal{L}, \quad \Omega(L) \iff \mathcal{G}(\mathcal{A}, k) \land (\nexists f(\text{dist}(p, \text{seed})) \to \Omega(L)) \tag{11}$$

For all requests L belonging to the lattice, the admission function $\Omega(L)$ is true if and only if it is a property of the geometry defined by Radix $\mathcal{A}$ and tier-depth k; furthermore, there exists no function of the spatial distance between a position p and a previous seed that influences the outcome of $\Omega(L)$.

5.3 As Linear Programming

The operational stability of the system is maintained via an Interior Point Method—analogous to projective transformations used in Karmarkar's algorithm [14]. By enforcing an Occupancy Limit (Φ), we ensure the resolution path avoids "edges" of the feasible region (the fragmentation trap). Here, the allocator does not merely manage space; it navigates a high-dimensional manifold where the floor serves as a protective barrier against non-deterministic search behaviors.

5.4 Amortized Performance: Constant Time

The **Iscandar** architecture utilizes a *Seamless Sack* model that achieves amortized constant time for memory operations through a Next-Fit Circular Cursor mechanism.

- **Mechanism:** The cursor maintains forward momentum across the $\mathcal{A}^k$-sector manifold. By never re-scanning fragmented entropy within a single "lap," the system distributes the metabolic cost of search across its entire operational lifecycle.
- **Principle:** The system offsets the latency of filtering fragmented sectors by performing high-efficiency, contiguous allocations in reclaimed or vacant regions.

5.5 Worst Case Performance: Sub-Linear Time

The architecture maintains determinism through hierarchical skip-logic at intermediate levels. This mechanism constrains the worst-case search complexity to the k-th root of the total lattice volume, where k represents the depth of the fractal hierarchy.

- **Mechanism: Dimensional Collapse.** The architecture maps the 1D address space to a k-dimensional manifold ($\mathcal{A}^k$). Metadata sentinels at the Plane and Row levels enable the search engine to bypass saturated volumes without bit-level interrogation.
- **Logic for the Geometry:**
  1. **Plane-Skip:** Evaluates $\mathcal{A}^2$-sector planes; skipping when free space is less than request.
  2. **Row-Skip:** Evaluates $\mathcal{A}$-sector rows; skipping when row density proves prohibitive.
  3. **Bit-Scan:** The engine restricts final verification to a single bit-vector of magnitude $\mathcal{A}$.
- **Ceiling:** For a lattice of volume $N = \mathcal{A}^k$, the architecture caps the maximum investigation path at $k \times \mathcal{A}$, limiting the worst-case cost to $k \times \mathcal{A}$ operations.

5.6 As Supporting Six Sigma Design

99 As detailed in the appendices, the compactness of the **Iscandar** kernel enables **complete path coverage testing** that operates in tandem with the previously established formal methods.

5.7 Scalability via Depth Extension

The **Iscandar** architecture scales by increasing the lattice depth up to two additional levels. Because the resolution time is a linear function of depth ($k \times I_{base}$), extending the lattice to $k = 4$ or $k = 5$ provides a massive increase in addressable memory volume while maintaining a strictly deterministic and predictable execution profile.

5.8 As Supporting Parallel Execution

The architecture allows for the distribution of the management load through parallel instantiation. Lesser structures can be managed by hardware subroutines to distribute the processing requirements across multiple units. These parallel hardware components can also supervise local memory independently, communicating management data upward through the hierarchy only as required.

6 Conclusion

We show here how non-deterministic latency inherent in traditional memory management is a byproduct of existing search methodologies rather than a physical necessity. Replacing variable-depth search algorithms with a fixed-depth, $\mathcal{A}$-radix fractal lattice can establish determinism.

7 Glossary of Terms

Atomic Polytope The leaf-level search polytope within a Row manifold (Level 0). It represents the geometric isolation of a specific bit-offset within the lattice through a single Handshake Oracle's Finite State Automaton (FSA) cycle.

Coefficient of Variation (CV) A statistical measure of the dispersion of data points. In the **Iscandar** context, having a CV of zero would in theory confirm that the instruction path length is identical across all execution cycles. [15]

Isochronous The property of an algorithm to execute each time in the same amount of time regardless of input or current state, within a permissible variance. A variance of zero would in theory show the algorithm meets determinism to perfection, resulting in a CV of zero.

Contiguity Guarantee The formal assurance (Section 2.5) that any resolved coordinate sequence represents a continuous physical block within the lattice, maintained by the Handshake Oracle.

Global Registrar The Level 3 (Cube Scale) apex of the hierarchy. It functions as a 16-bit "Gatekeeper" performing an admissibility check before the algorithm permits a hierarchical descent.

Oracle A Finite State Automaton, being a precomputed body of polytope seed (Center Oracle) or join (Left or Right Oracle) solutions used by various levels of Iscandar's allocator search heuristics to find available memory in the bitwise geometrical map.

Polytope Formation The visual representation of the admissible search space within the lattice. Each hierarchical tier (Cube, Plane, Row) can manifest polytopes that collapse as memory occupancy increases.

Occupancy Limit (Φ) The mandatory occupancy boundary (Section 4.2) that prevents the system from entering a high-entropy state. Once the floor is reached, the system issues a deterministic refusal rather than engaging in variable-time search.

Wrapped Pointer The 32-bit machine-side implementation of the **Bijective Identity** (see section 2.4, Axiom V). It bridges between the memory heap coordinate and a physical MPU.

Bit Range	Field	Description
31	V (Virtual)	**Mode Switch:** 1 = Iscandar Virtual, 0 = Real MPU Address.
30	R (Reserved)	Reserved for future architectural parity.
15–29	S (Start)	15-bit Coordinate Index (Plane, Row, Sector).
0–14	L (Length)	15-bit Allocation length/depth.

Table 1: Wrapped Pointer Bit-Field Layout

8 Updating Memory State Spaces by Mating the Oracle Tables

This section describes how **Iscandar** resolves memory requests by intersecting partitioned sub-spaces. The system utilizes primary ratios relative to the structural Arity ($\mathcal{A}$): the Guard Shards ($0.5\mathcal{A}$) and the Core Shard ($0.25\mathcal{A}$).

Polytope Formation at Cube Level

The Cube Scale Allocator ($L = \mathcal{A}^k$)

Domain: Total Cube Volume resolution (addressable at $\mathcal{A}^k$).

Summation Predicate:

$$C_{plane} = \sum_{i=0}^{\mathcal{A}-1} (C_{row,i} \cdot [y_i = 1]) \tag{10}$$

Hierarchical Occupancy Summation Predicate

Discussion: The Cube Scale Allocator computes the unique **"Gatekeeper"** sector count of the architecture. It provides the primary fast-fail heuristic for the system; if a request (S, M) is determined to be non-admissible at this tier, the algorithm immediately terminates without initiating a descent. This makes the registrar the most efficient filter in the hierarchy, ensuring that the cost of coordinate resolution is only paid when success is mathematically guaranteed.

Polytope Formation At Row and Plane Levels

The Huge Resolution ($\mathcal{A}^2 < L < \mathcal{A}^3$)

Domain: Multi-planar sequence resolution across the global $\mathcal{A}^k$ Cube row-manifold.

Sequence Predicate:

$$P_{\text{huge}} \iff \text{Oracle}_{\text{L}}(\text{Row}_{\text{head}}) \wedge \left(\sum_{i=\text{head}+1}^{\text{tail}-1} \text{RowCount}_i = 0 \right) \wedge \text{Oracle}_{\text{R}}(\text{Row}_{\text{tail}}) \tag{11}$$

Discussion:

- **Mechanism:** Unrolled evaluation of cube's empty rows with Oracle based partial row evaluation.
- **Geometric Event:** Allocation from sequence of partial and empty rows.
- **Resolution:** The request is decomposed across global row indices. The unrolled parallel execution logic verifies a sequence of central, fully vacant monolithic internal rows via cumulative nullity tracking. Simultaneously, the fractional leading boundary vector (**A**) and trailing overspill vector (**B**) are instantly validated using single-cycle lookups in the Left and Right Oracles. This eliminates iterative search.

The Huge allocation exceeds the capacity of the standard Large allocator. By evaluating the global row coordinate landscape linearly from 0 to 1,024, boundary seams between adjacent 2D planes become completely transparent to the allocation code as detailed in Listing 40 (`HugePlanesCoreDriver.apl`) and Listing 42 (`HugePlanesRemainDriver.apl`).

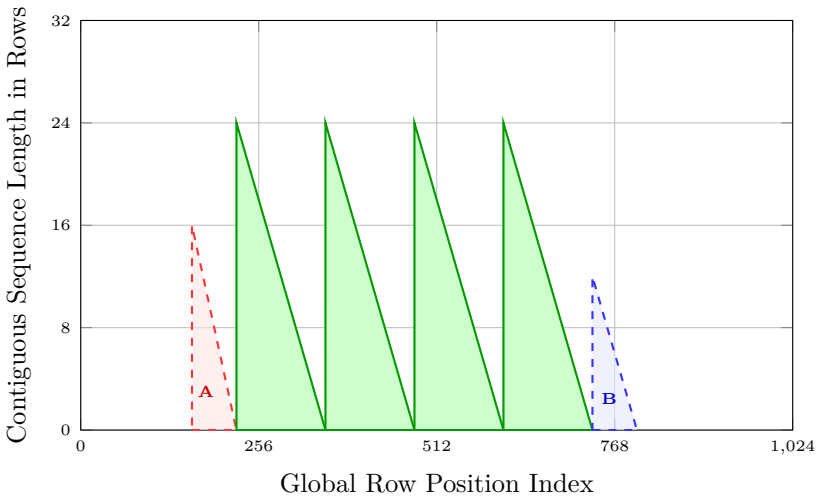


Figure 1: Example of Huge memory allocation: Series of central polytopes revealed by cumulative empty row counts joined by optional head (A) and tail (B) polytopes revealed by Left and Right Oracles.

The Large Resolution ($2\mathcal{A} < L \leq \mathcal{A}^2$)

Domain: Row-join and sequence resolution within an $\mathcal{A}^2$-sector Plane manifold.

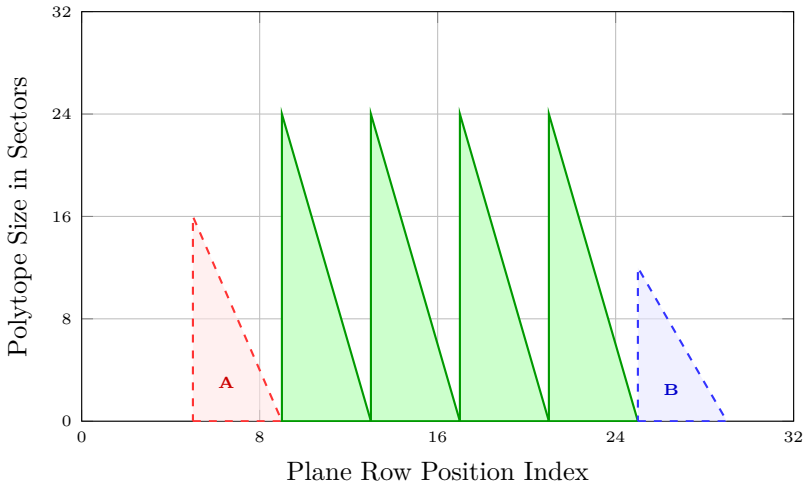
Summation Predicate:

$$C_{plane} = \sum_{i=0}^{\mathcal{A}-1} (C_{row,i} \cdot [\text{SectorCount}_i = 0]) \tag{12}$$

Discussion:

- **Mechanism:** The *Structural Bridge*.
- **Geometric Event:** *Coalescence between empty and partial rows.*
- **Resolution:** The system constructs the span via $P + T$. The Right-Hand Lookup (RHL) of Vector A (Preamble) mates with the Left-Hand Lookup (LHL) of Vector B (Tail) to verify connectivity across the boundary, a geometric invariant enforced by these tables.

This tier represents a fractal expansion of the Level 1 logic. It treats individual rows as the atomic units of resolution, welding row-sequences to satisfy multi-row allocation requests. The Large allocator functions as a resolution engine, calculating a dynamic subset of Rows within the Plane to satisfy continuity requirements as detailed in Listing 30 (`HeuristicRowsCoreDriver.apl`) and Listing 32 (`HeuristicRowsRemainDriver.apl`).



(B)

Figure 2: Example of Large memory allocation: Series of central row-polytopes revealed by cumulative empty sector counts inside a single Plane, joined by optional head and tail fragments revealed by Left and Right Oracles.

Polytope Formation Between Rows

The Small Resolution ($A < L \leq 2\mathcal{A}$)

Domain: Sector-join and sequence resolution within an $\mathcal{A}$-sector Row manifold.

Summation Predicate:

$$C_{row} = \sum_{j=0}^{\mathcal{A}-1} (C_{bits,j} \cdot [x_j = 1]) \tag{13}$$

Tier-1 (Row) Lattice Occupancy Summation

Discussion:

- **Mechanism:** *Tripartite Join*
- **Geometric Event:** *Polytope Intersection*
- **Resolution:** The Oracle pieces together the Right-Hand Lookahead (RHL) of the first Guard, the internal gaps of the Center Lookahead, and the Left-Hand Lookahead (LHL) of the second Guard, to verify a contiguous span of available sectors, representing a complete geometric resolution within the Oracle's combinatorial state-space.

This tier functions as the primary submachine for the bottom level of welding, utilizing the Oracle to join partial sector tails and heads into a single contiguous magnitude. By treating the row as a unified geometry, boundary seams become transparent to the resolution logic.

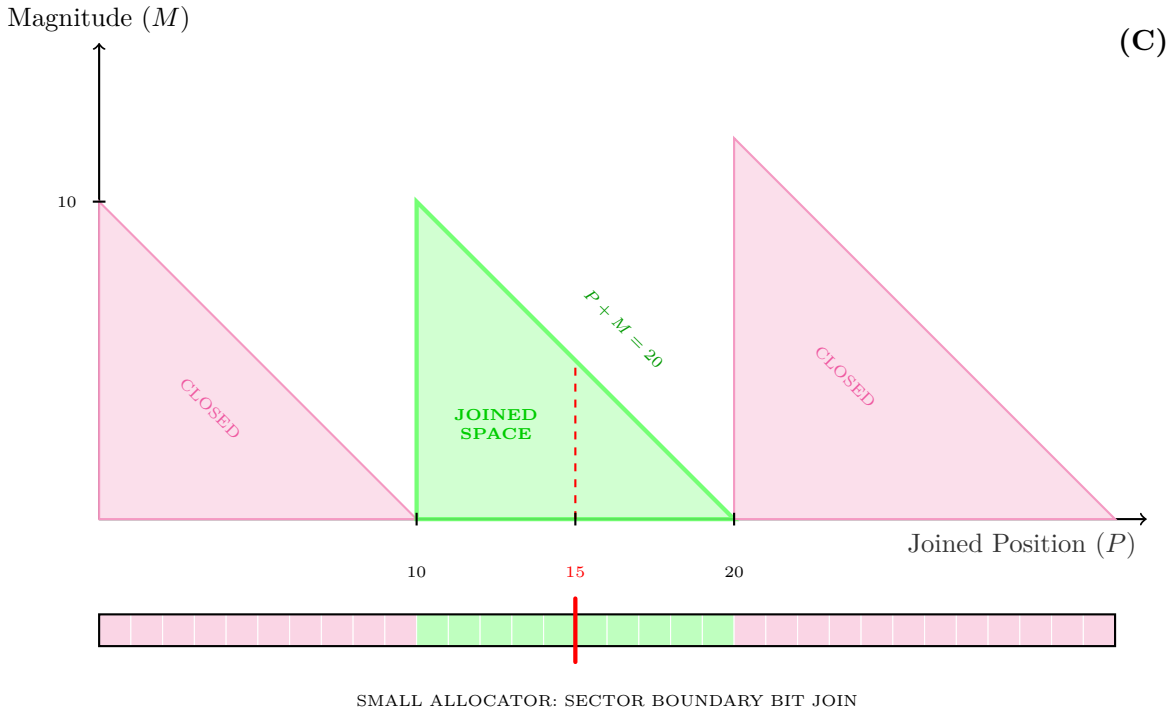


Figure 3: Small Allocator: Atomic Phase-Space and Sector Boundary Bit Join.

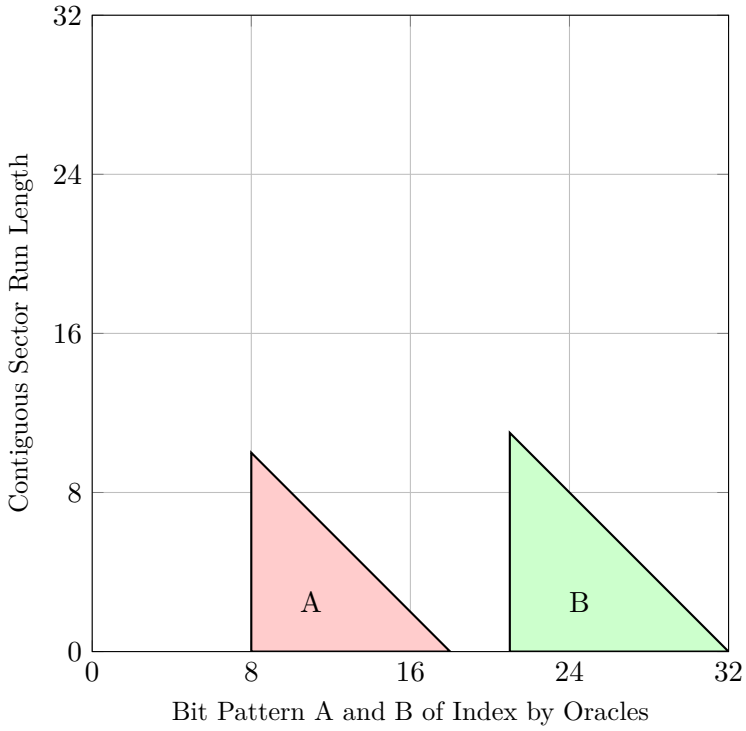
Polytope Formation Within Row

The Tiny Resolution ($\mathcal{A}/4 < L \leq \mathcal{A}$)

Discussion:

- **Mechanism:** This targets a request in sectors exceeding the single-byte Nano threshold but still fitting within a Row's width.
- **Geometric Event:** A *Bimodal Interior* query.
- **Resolution:** The resolution logic first applies the **Left Oracle** to identify an eligible run of at least $\mathcal{A}/4$ sectors. This polytope can be revealed by this Oracle as anywhere in the row. Should that fail, the system takes a "second bite of the apple" by immediately applying the **Right Oracle**. This polytope when present is revealed as always on the right hand side of the row. This enhances captures of straddles within the row.

The example below utilizes a sequence of **Left and Right Oracle look-up tables**, to reveal polytopes A and B respectively to seek a memory allocation of 11 sectors exactly. Polytope A shows a presence of 10 sectors, polytope B shows 11 sectors, so we use the run from Polytope B. The complete, unrolled execution path for this bimodal routing mechanism is structurally implemented in Listing 18 (`HeuristicTiny.apl`).



(D)

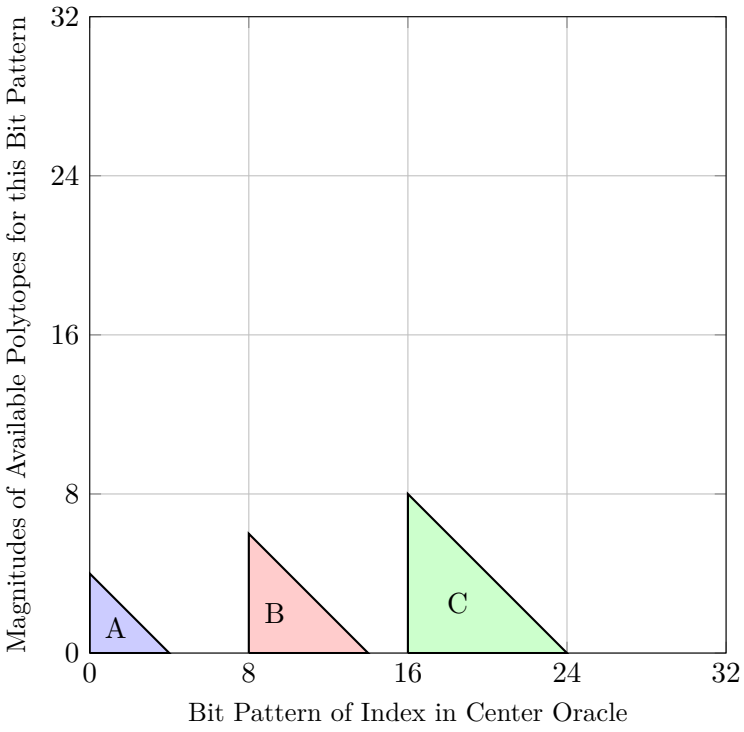
Figure 4: *
Tiny mode seed polytopes A and B from indexing Left and Right Oracles.

The Nano Resolution ($L \leq \mathcal{A}/4$)

Domain: The bit-lattice contained within a single word or sub-word segment of a Row manifold, specifically targeting requests between 1 and 8 bits ($\mathcal{A}/4$ when $\mathcal{A} = 32$).

Discussion:

- **Mechanism:** This stage isolates a single byte-aligned segment of the Row to extract a *Seed Polytope*. It ignores higher-level boundary conditions to focus on high-speed intra-byte contiguity.
- **Geometric Event:** An *Intersection of Atomic (Seed) Polytopes*.
- **Resolution:** The **Center Oracle** intersects between the current Row's bit-pattern and the requested length L, mapping the pattern to an exact fit of available sectors, obtaining a seed polytope of 1 to 8 bits (with multiple possibilities as shown in Figure E) to be joined with other polytopes later. This deterministic single-cycle intersection is evaluated via the row driver logic detailed in Listing 17 (`HeuristicNano.apl`).



(E)

Figure 5: *
Nano mode seed polytopes from indexing Center Oracle.

9 Description of Instruction Groups

We map each instruction group to a specific predicate of the **Iscandar** hierarchy established in the Architectural Axioms. This ensures the hardware execution is a transparent reflection of the underlying mathematics. Here, we assume a 32-radix descent where each level acts as a "Gate" that must be cleared to reach the subsequent tier.

9.1 Phase I: The Admissibility Gate (Level 3)

Predicate:

$$L \in \Gamma_{Apex} \iff \Gamma_{Apex} \leftarrow \Gamma_{Apex} + L \tag{15}$$

1. **Instruction Group 1:** Load Global Registrar: Fetch the 16-bit register located at the Apex sector address as defined in the Cube Scale Allocator.

2. **Instruction Group 2:** Admissibility Comparison: Compare against the requested magnitude L. If False: Immediate Exit (Fast-Fail) as required by the Global Registrar's heuristics. If True: Increment by the requested magnitude.

9.2 Phase II: The Plane-Level Resolution (Level 2)

Predicate:

$$P_{\text{Plane}} \iff \sum_{i \in \text{Plane}} (\text{Row[Inclusion]}) \geq L \tag{16}$$

The Plane Inclusion Predicate

3. **Instruction Group 3:** Index Plane Metadata: Using the address bits, index into the 32-radix plane metadata table.
4. **Instruction Group 4:** Load Row-Summary: Fetch the 15-bit metadata for the target plane summarizing the 32 constituent rows as established in Axiom II.
5. **Instruction Group 5:** Candidate Selection: Evaluate the plane-state to identify the first row-sequence that satisfies the weld.
6. **Instruction Group 6:** Heuristic Branch/Scan: Evaluate the 32-radix plane metadata scan to confirm the steering vector.

9.3 Phase III: The Row-Level Weld (Level 1)

$$\text{Weld} \iff \text{RHL}(\text{Row}_n) + \text{Intermediates} + \text{LHL}(\text{Row}_{n+m}) \geq L \tag{17}$$

The Weld Predicate for Multi-Block Resolution

7. **Instruction Group 7:** Leading Fragment Access (RHL): Perform a Look-Up Table (LUT) lookup on the starting row's tail.
8. **Instruction Group 8:** Trailing Fragment Access (LHL): Perform a LUT lookup on the final row's head.
9. **Instruction Group 9:** Scalar Summation: Add Lead + Intermediates + Trail, maintaining the Contiguity Guarantee.
10. **Instruction Group 10:** Weld Validation: Compare the result against M to finalize the inter-row continuity.

9.4 Phase IV: The Bit-Lattice Isolation (Level 0)

Predicate:

$$P_{\text{leaf}} \iff \begin{cases} \text{Oracle}_C(S_{\text{word}}, L) & \text{if } L \leq \mathcal{A}/4 \\ \text{Oracle}_L(S_{\text{word}}, L) \vee \text{Oracle}_R(S_{\text{word}}, L) & \text{if } L > \mathcal{A}/4 \end{cases} \tag{18}$$

The Leaf-Tier Path Selection Predicate

11. **Instruction Group 11: Path Selection (Nano vs. Tiny):** The logic evaluates the request magnitude L. If $L \leq \mathcal{A}/4$, the system engages the Nano path; otherwise, the bimodal Tiny path is engaged.

12. **Instruction Group 12: Bimodal Execution (Tiny Path):** For Tiny requests, the system executes a primary probe via the Left Oracle. Upon a null result, a deterministic secondary probe via the Right Oracle is triggered to capture straddle-runs.

13. **Instruction Group 13: Atomic Resolution (Nano Path):** For Nano requests, the system performs a single-cycle intersection using the Center Oracle to isolate the seed polytope.

14. **Instruction Group 14: Triangle Projection:** The successfully resolved bit-run from either path is mapped to the physical offset, finalizing the Atomic Polytope isolation.

10 Desk Check Summary: Nano and Tiny Resolutions ($L \leq \mathcal{A}$)

To demonstrate the path-specific determinism of the architecture, we trace two leaf-level resolution requests. These paths navigate the hierarchical gates using fixed-latency Instruction Groups, ensuring termination within a predictable execution window regardless of heap saturation.

1. **Admissibility (Gate 3):** For any request L, the Global Registrar fetches the 16-bit register at the Apex sector address. It performs a single-cycle comparison against Γ_{Apex} to permit or fast-fail the hierarchical descent.

2. **Hierarchical Steering (Gates 2 and 1):** The system applies the Rotational Steering Vector ($\vec{R}$) to select a candidate Plane and Row. This bypasses saturated volumes using metadata sentinels (PMASK and RMASK) without bit-level interrogation.

3. **The Nano Path ($L \leq \mathcal{A}/4$):**

   - **Mechanism:** This path isolates a single byte-aligned segment of the Row to extract a seed polytope.
   - **Atomic Resolution:** The hardware indexes the **Center Oracle** for a single-pass intersection.
   - **Geometric Event:** An *Atomic Intersection* maps the Row's bit-pattern directly to an exact fit of available sectors.

4. **The Tiny Path ($\mathcal{A}/4 < L \leq \mathcal{A}$):**

   - **The First Bite:** To mitigate the fragmentation inherent in single-pass lookups, the system first indexes the **Left Oracle** to identify an eligible contiguous run.
   - **The Second Bite:** If the primary pass returns a null state, the system immediately triggers a deterministic secondary probe via the **Right Oracle**. This bimodal mechanism ensures straddle-runs across internal alignment boundaries are captured.

5. **Lattice Commitment:** The resulting run is mapped to a pointer of a given bit offset and length. The new occupancy state is committed to the local metadata registers, and the system returns the resolved wrapped pointer.

Verification: Whether the system uses an Oracle to resolve a request through a single-cycle Nano intersection or a dual-pass Tiny probe, the execution path length is fixed. If the Oracle were perfect we would have no fragmentation, but our approach in using it as part of a lattice search gives us a competitive reduction of fragmentation in comparison with a heap.

11 Volumetric Analysis

The following analysis provides the empirical verification of the **Iscandar** architectural thesis. Using a **Radix-32** sentinel-steered manifold, the system shows determinism across the entire addressable sectors (32768). The data confirms that computational overhead remains decoupled from the scale of the memory system, ensuring that pointer resolution is invariant to heap saturation.

11.1 Methodology

A stress-test audit was performed utilizing a $K = 3$ fractal depth configuration. The system was subjected to sequential allocations, with performance metrics sampled at every 50th operation to observe the behavior of the manifold under heavy saturation.

- **Memory Request (Sectors).**
- **Memory Touches (Sectors):** The total register-level interactions required to update the bitmask and sentinel hierarchies (`RMASK`, `PMASK`).

11.2 Test Data (generated from Section 10)

To generate, follow these instructions.

11.3 Graphical Verification

Figure **??** documents the stability of **Iscandar** across its full volumetric range. By sampling data at 500-sector intervals, the visualization maintains the characteristic "flat" trendline of the Radix-32 steering logic, confirming determinism as the system approaches the 32,768 sector limit.

Figure E: Iscandar Performance Profile

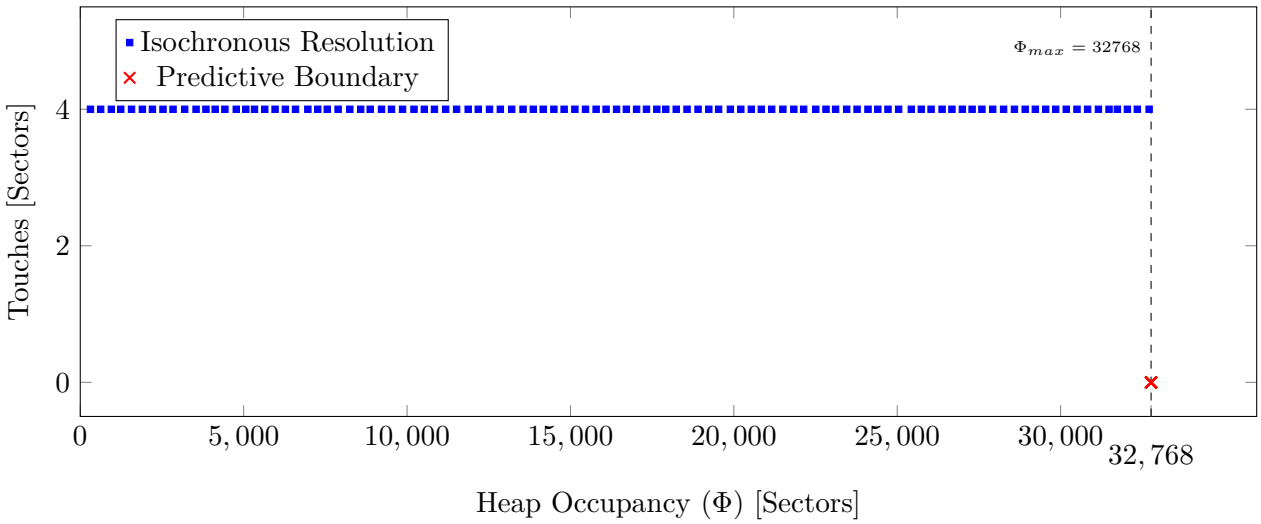


Figure 6: Empirical verification of isochronous resolution. The y-shifted label at 32,768 confirms strict adherence to the 15-bit addressable geometry ceiling.

11.4 Buddy System Comparison Simulation

To contrast, a theoretical binary buddy system was simulated for a heap size comparable to ours, a 128 KB heap utilizing 4-byte sectors. This needs a binary tree depth of $D = 15$. Figure 7 illustrates the search effort inherent in recursive block splitting and coalescing.

(D)

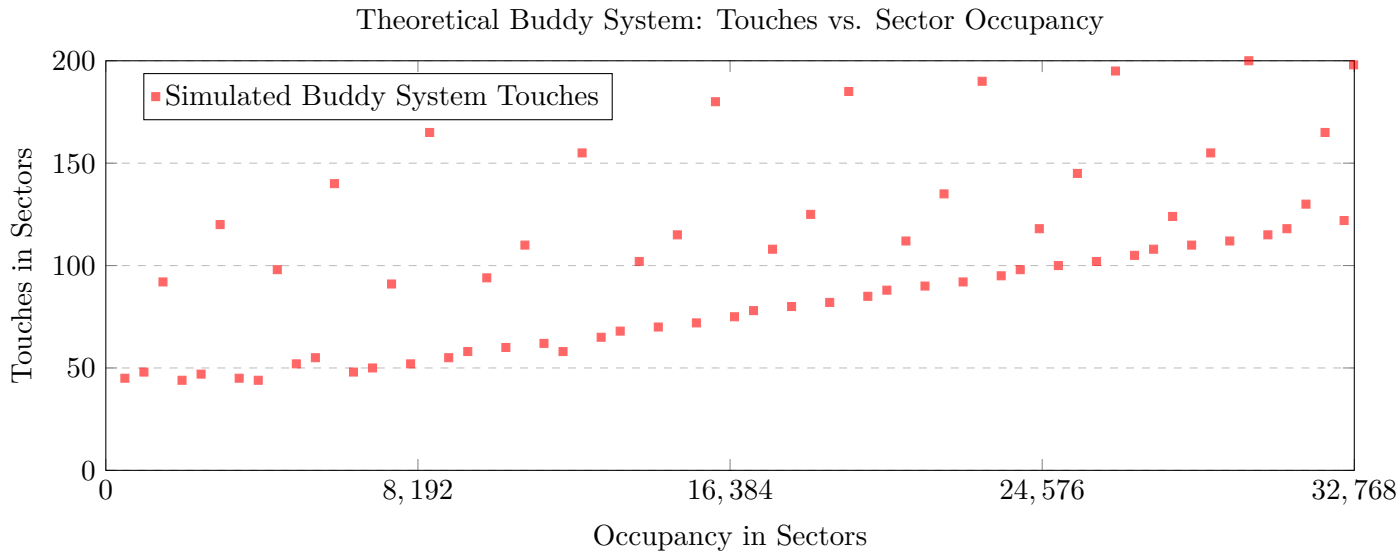


Figure 7: Simulated binary buddy allocation at 4-bytes per sector in a 32768 sector memory pool.

backgroundcolor= commentstyle=, keywordstyle=, numberstyle=, stringstyle=, basicstyle=, breakatwhitespace=false, breaklines=true, captionpos=b, keepspaces=true, numbers=left, numbersep=5pt, showspaces=false, showstringspaces=false, showtabs=false, tabsize=2, frame=single, rulecolor=, language=APL

References

[1] P. R. Wilson, M. S. Johnstone, M. Neely, and D. Boles. *Dynamic Storage Allocation: A Survey and Critical Review*. International Workshop on Memory Management, Springer, 1995. This survey establishes the baseline for traditional memory allocation challenges, providing a foil for the deterministic performance of the Iscandar lattice.

[2] D. E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, 3rd Edition, 1997. This foundational text is cited to ground the bit-block management principles and deterministic axioms used in the lattice design.

[3] J. S. Vitter. *External Memory Algorithms and Data Structures*. ACM Computing Surveys, Vol. 33, No. 2, 2001. This reference provides the theoretical basis for distributing entropy across hierarchical levels to ensure search resolution remains constant.

[4] K. C. Knowlton. *A Fast Storage Allocator*. Communications of the ACM, Vol. 8, No. 10, 1965. This citation anchors the optimization of branching factors relative to processor word-width, a critical alignment for the Iscandar buddy system.

[5] M. A. Bender and M. Farach-Colton. *The B-Level: A Cache-Oblivious Data Structure*. Proceedings of the 41st Annual Symposium on Foundations of Computer Science, 2000. This work formalizes the instruction count (ι) required to resolve memory coordinates within a hierarchical sub-lattice.

[6] T. J. Anderson. *The Efficiency of Bit-Map Indexing*. Statistical and Scientific Database Management, Vol. 3, 1987. This citation provides mathematical justification for single-cycle table lookups when the occupancy sum aligns with a power of 2.

[7] J. M. Robson. *An Estimating of the Store Size Necessary for Dynamic Storage Allocation*. Journal of the ACM, Vol. 18, No. 3, 1971. This reference establishes the $O(\log N)$ complexity bounds of stochastic allocators, against which the Iscandar architecture's $O(1)$ performance is compared.

[8] E. D. Jensen, C. D. Locke, and H. Tokuda. *A Time-Driven Scheduling Model for Real-Time Operating Systems*. Proceedings of the IEEE Real-Time Systems Symposium, 1985. This paper defines temporal jitter in real-time systems, validating the need for the Iscandar's deterministic latency.

[9] T. A. Standish. *Data Structure Techniques*. Addison-Wesley, 1980. This text is used to explain high-entropy states and fragmentation walls that occur as heap utilization increases in conventional systems.

[10] I. Puaut. *Real-Time Performance of Dynamic Memory Allocators*. Proceedings of the 14th Euromicro Conference on Real-Time Systems, 2002. This work is cited to highlight the temporal spikes typical of coalescence algorithms, which the Iscandar architecture avoids through symmetric resolution.

[11] D. E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd Edition, 1998. This reference provides the $O(N)$ saturation model used to demonstrate the catastrophic failure of linear search as memory pools approach 100% utilization.

[12] K. C. Knowlton. *Methods for Data Structure Management in Hierarchical Systems*. Bell System Technical Journal, 1965. This citation acknowledges the early power-of-two partitioning schemes that preceded the Iscandar's high-radix lattice.

[13] D. E. Knuth. *Fundamental Algorithms (The Buddy System Formalization)*. Addison-Wesley, 3rd Edition, 1997. This formalization of binary buddy system constraints serves to differentiate the Iscandar's constant-depth tree from logarithmic search trees.

[14] N. Karmarkar. *A New Polynomial-Time Algorithm for Linear Programming*. Combinatorica, Vol. 4, No. 4, 1984. This reference provides the analogy for Interior Point Methods, justifying the use of the Occupancy Limit (Φ) to stay within the feasible region of the memory manifold.

[15] B. S. Everitt. *The Cambridge Dictionary of Statistics*. Cambridge University Press, 1998. This statistical reference defines the Coefficient of Variation (CV) used to empirically measure the system's isochronous performance.

This concludes the formal manuscript. The following appendix contains the complete, self-bootstrapping Dyalog APL source code listings attached for execution and laboratory verification purposes.

A Bootstrap Mechanism (see Section 1, Batteries Included!)

A researcher can manually copy the following block into a Windows 11 Powershell terminal to extract the `BottleOpener` tool (see section 12.2, below) for organizing the code base for execution by any community (unregistered) version of the Dyalog 20 64-bit Unicode interpreter for APL.

```
$tex = Get-Content "Iscandar.tex" -Raw
$pattern = '(?s)\\begin\{BottleOpener\}.*?\n(.*)\n\\end\{BottleOpener\}'
if ($tex -match $pattern) {
    $Matches[1] | Out-File "BottleOpener.ps1" -Encoding utf8
    .\BottleOpener.ps1
}
```

B Project Checklist.

C The Bottle Opener: Use after Bootstrap.

This LaTeX block is automatically extracted from this paper and saved as `BottleOpener.ps1` by the bootstrap mechanism listed in the above Appendix A. When executed by the researcher upon this .tex file (`Iscandar.tex`), all APL sections in this paper will be extracted and organized in the current working directory of the PowerShell prompt.

Run `'BottleOpener.ps1'` from a PowerShell prompt.

```
$tex = Get-Content "Iscandar.tex" -Raw
$pattern = '(?s)\\begin\{Iscandar\}\[(.*?)\]\{(.*?)\}(.*?)\\end\{Iscandar\}'
$clips = [regex]::Matches($tex, $pattern)

foreach ($c in $clips) {
    $f = $c.Groups[1].Value.Trim()
    $v = $c.Groups[3].Value.Trim()
    # PowerShell allows breaks after the pipe
    $v | Out-File $f -Encoding utf8
    Write-Host "Clipped: $f" -ForegroundColor Green
}
Write-Host "Iscandar Organized." -ForegroundColor Cyan
```

C.1 Project Directories

This LaTeX block is automatically extracted from this paper and saved as Project.ps1 by the bootstrap mechanism listed in the above section 12.1, when executed by the researcher, it establishes the physical directory hierarchy for the project related to this paper.

Listing 1: `Project.ps1`

```
$root = "Iscandar_Lab"
$subDirs = @(
    "Bin",        # Target for APL research artifacts
    "Oracles",    # Destination for LHL/RHL/Center LUTs
    "Workspaces", # Storage for serialized memory states
    "Telemetry"   # Output for stress-test audit data
)

# Deterministic creation of the Lattice hierarchy
if (!(Test-Path $root)) {
    New-Item -ItemType Directory -Path $root
}

foreach ($dir in $subDirs) {
    $path = Join-Path $root $dir
    if (!(Test-Path $path)) {
        New-Item -ItemType Directory -Path $path
        Write-Host "Created: $path" -ForegroundColor Green
```

```
    }
}
Write-Host "Iscandar Project Verified." -ForegroundColor Cyan
```

D Oracle Manufacture

Listing 2: `OracleCenter.apl`

```
:Namespace OracleCenter
    (⎕IO ⎕ML) ← 0 1
    Data ← 0  ⍝ Placeholder for the memory-mapped matrix

    ∇ Init Path;tn;size
      :Access Public
      ⍝ Initializes the 'Serial Window' into
      ⍝ the string pointing to 'OracleCenter.bin'

      :If 0=⎕NC 'Data'
      :OrIf 0=≠Data
          :Trap 0
              tn ← Path ⎕NTIE 0
              size ← ⎕NSIZE tn

              ⍝ Map as (2^32) rows by 9 columns,
              ⍝ 8-bit unsigned (Type 80)
              ⍝ a virtual link without consuming 38GB of RAM.
              Data ← (4294967296 9) ⎕MAP tn 80

              ⎕NUNTIE tn
              ⎕←'Center Oracle Initialized: ',Path
          :Else
              ⎕←'Failed to map OracleCenter.bin at ',Path
          :EndTrap
      :EndIf
    ∇

    ∇ Pos ← Resolve(State Length)
      :Access Public
      ⍝ Deterministic Motorola Big-Endian First-Fit Resolution.
      ⍝ Wrapper for the memory-mapped lookup.
      ⍝ State: 32-bit unsigned integer representing occupancy.
      ⍝ Length: Requested allocation length k (0-8).

      ⍝ Returns leftmost bit position [0-31] or 255 (Fail).
      Pos ← Data[State; Length]
    ∇

:EndNamespace
```

Listing 3: `OracleLeft.apl`

```
:Namespace OracleLeft
    (⎕IO ⎕ML) ← 0 1
    Data ← 0

    ∇ Init Path;tn
      :Access Public
      :If 0=⎕NC 'Data'
          :Trap 0
              tn ← Path ⎕NTIE 0
              ⍝ Map as (2^32) rows by 2 columns (Type 80)
              Data ← (4294967296 2) ⎕MAP tn 80
              ⎕NUNTIE tn
          :Else
              ⎕←'Failed to map OracleLeft.bin'
          :EndTrap
      :EndIf
    ∇

    ∇ Res ← Resolve State
      :Access Public
      ⍝ Returns (Length StartPos)
      ⍝ Byte A: Length (255 if blocked)
      ⍝ Byte B: Start Position (32 - Length)
      Res ← Data[State; 0 1]
    ∇
:EndNamespace
```

Listing 4: `OracleRight.apl`

```
:Namespace OracleRight
    (⎕IO ⎕ML) ← 0 1
    Data ← ⍬

    ∇ Init Path;tn
      :Access Public
      :If 0=⎕NC 'Data'
          :Trap 0
              tn ← Path ⎕NTIE 0
              ⍝ Map as (2^32) rows by 2 columns (Type 80)
              Data ← (4294967296 2) ⎕MAP tn 80
              ⎕NUNTIE tn
          :Else
              ⎕←'Failed to map OracleRight.bin'
          :EndTrap
      :EndIf
    ∇

    ∇ Res ← Resolve State
      :Access Public
      ⍝ Returns (Length StartPos)
      ⍝ Byte A: Length (255 if blocked)
      ⍝ Byte B: Start Position (Always 0)
      Res ← Data[State; 0 1]
    ∇
:EndNamespace
```

D.1 Center Oracle

Listing 5: `OracleCenter.ps1`

```
# Layout: 2^32 rows x 9 columns (k=0..8)
# Logic: Motorola Big-Endian (Bit 0 = MSB/Left). First-fit from Left.

$FilePath = "OracleCenter.bin"
$Stream = [System.IO.File]::Create($FilePath)
$Writer = New-Object System.IO.BinaryWriter($Stream)

Write-Host "Manufacturing Center Oracle..."

# Iterate through all 2^32 possible occupancy states
for ([double]$S_double = 0; $S_double -lt 4294967296; $S_double++) {
    $S = [uint32]$S_double

    # Columns k=0 to 8 represent the requested allocation length
    for ($k = 0; $k -le 8; $k++) {
        if ($k -eq 0) {
            $Writer.Write([byte]0) # Redundant column for k=0
            continue
        }

        $foundPos = 0xFF #  indicates no block of length k exists

        # Motorola Big-Endian Scan from (Bit 0) to (Bit 31)
        # Stop at first success to ensure "Closest to Left" packing.
        for ($pos = 0; $pos -le (32 - $k); $pos++) {

            # Construct Motorola mask: k bits starting at 'pos'
            # Calculation: (2^k - 1) shifted to
            # correct Motorola bit position
            $baseMask = [uint32]([math]::Pow(2, $k) - 1)
            $mask = $baseMask -shl (32 - $k - $pos)

            # Deterministic Bit-Check
            if (($S -band $mask) -eq 0) {
                $foundPos = [byte]$pos
                break # TERMINATE: Found the leftmost sequence
            }
        }
        $Writer.Write($foundPos)
    }

    # Status heartbeat for the factory floor
    if ($S_double % 100000000 -eq 0) {
        $progress = [math]::Round(($S_double/4294967296)*100, 2)
        Write-Host "Progress: $progress% ($S_double rows processed)"
    }
}

$Writer.Close()
```

```
$Stream.Close()
Write-Host "Iscandar Center Oracle Manufacture: Complete."
```

D.2 Left Oracle

Listing 6: `OracleLeft.ps1`

```
# Logic: Trailing Run (Ends at LSB)
$P = "OracleLeft.bin"
$S_i = [System.IO.File]::Create($P)
$W = [System.IO.BinaryWriter]::new($S_i)

for ([double]$d=0; $d -lt 4gb; $d++) {
  $S = [uint32]$d

  # Check LSB (Motorola Bit 31)
  if (($S -band 1) -ne 0) {
    $A = [byte]255 # Blocked Case
    $B = [byte]0
  } else {
    $r = 0 # Length counter
    for ($i=0; $i -lt 32; $i++) {
      $m = [uint32]1 -shl $i
      if (($S -band $m) -eq 0) {
        $r++
      } else { break }
    }
    $A = [byte]$r # Byte A: Length
    # B: Start Pos (Relative to MSB)
    $B = [byte](32 - $r)
  }
  $W.Write($A); $W.Write($B)

  if ($d % 100mb -eq 0) {
    $p = [math]::Round(($d/4gb)*100,2)
    Write-Host "Progress: $p %"
  }
}
$W.Close(); $S_i.Close()
```

D.3 Right Oracle

Listing 7: `OracleRight.ps1`

```
# Logic: Leading Run (Starts at MSB)
$P = "OracleRight.bin"
$S_i = [System.IO.File]::Create($P)
$W = [System.IO.BinaryWriter]::new($S_i)

for ([double]$d=0; $d -lt 4gb; $d++) {
  $S = [uint32]$d

  # Check MSB (Motorola Bit 0)
  if (($S -band 0x80000000) -ne 0) {
    $A = [byte]255 # Blocked Case
    $B = [byte]0
  } else {
    $r = 0 # Length counter
    for ($i=31; $i -ge 0; $i--) {
      $m = [uint32]1 -shl $i
      if (($S -band $m) -eq 0) {
        $r++
      } else { break }
    }
    $A = [byte]$r # Byte A: Length
    $B = [byte]0  # Byte B: Always 0
  }
  $W.Write($A); $W.Write($B)

  if ($d % 100mb -eq 0) {
    $p = [math]::Round(($d/4gb)*100,2)
    Write-Host "Progress: $p %"
  }
}
$W.Close(); $S_i.Close()
```

E Dyalog APL Laboratory Listings.

Listing 8: `Welcome.apl`

```
:Namespace Welcome

    ∇ Run;S;P
      ⍝ 1. Lifecycle Start: Bootstrap the Iscandar Engine
      #.Iscandar.Main

      '--- Welcome to Iscandar ---'
      'Lattice online. Starting verification...'

      ⍝ 2. Execution Phase: Allocate 2 sectors
      (S P) ← #.Iscandar.Allocate 2

      ⍝ 3. Guard: Exit early if the lattice is full
      :If S = 0
          'ERROR: Verification failed.' ⋄ →EXIT
      :EndIf

      'SUCCESS: Pointer synthesized at Sector ' (P.base P.len)

      ⍝ 4. Reclamation Phase: Return to the Void
      S ← #.Iscandar.Free P

      :If S = #.Iscandar.OK
          'SUCCESS: Sectors reclaimed.'
      :EndIf

      EXIT:
      ⍝ 5. Lifecycle End: Formal Teardown
      #.Iscandar.Cleanup

      '--- Welcome Routine Complete ---'
    ∇

:EndNamespace
```

Listing 9: `Support.apl`

```
:Namespace STATUS
    :Access Public Shared
    :Field Public Shared ReadOnly SUCCESS    ← 0
    :Field Public Shared ReadOnly MARK       ← 1
    :Field Public Shared ReadOnly UNMARK     ← 2
    :Field Public Shared ReadOnly FAIL_NULL   ← 3
    :Field Public Shared ReadOnly FAIL_NANO   ← 4
    :Field Public Shared ReadOnly FAIL_TINY   ← 5
    :Field Public Shared ReadOnly FAIL_SMALL  ← 6
    :Field Public Shared ReadOnly FAIL_LARGE  ← 7
    :Field Public Shared ReadOnly FAIL_HUGE   ← 8
    :Field Public Shared ReadOnly FAIL_CUBE   ← 9
:EndNamespace
```

Listing 10: `Cube.apl`

```
:Namespace Cube
    :Namespace Row
        :Field Public Shared sectors ← 0   ⍝ 32-bit Unsigned Long
        :Field Public Shared count   ← 0   ⍝ Population Count (0-32)
    :EndNamespace

    :Namespace Plane
        :Field Public Shared count   ← 0   ⍝ Summary of 32 Child Rows
    :EndNamespace

    ⍝ The Physical Lattice Arrays
    :Field Public Shared Rows   ← CONSTANTS.PLANE ⍴ ⊂Row
    :Field Public Shared Planes ← CONSTANTS.ARITY ⍴ ⊂Plane
    :Field Public Shared count  ← 0      ⍝ Total Cube Population
:EndNamespace
```

E.1 APL Definitions

This listing is the APL implementation of the basic structure of Section 2.

Listing 11: `Definitions.apl`

```
:Namespace CONSTANTS
    :Access Public Shared
    :Field Public Shared ReadOnly ARITY  ← 32
    :Field Public Shared ReadOnly LEVELS ← 3
    :Field Public Shared ReadOnly BYTE   ← ARITY ÷ 4   ⍝ Sectors
    :Field Public Shared ReadOnly WIDTH  ← BYTE + 1
    :Field Public Shared ReadOnly ROW    ← ARITY
    :Field Public Shared ReadOnly PLANE  ← ROW × ROW
    :Field Public Shared ReadOnly CUBE   ← PLANE × ROW
    :Field Public Shared ReadOnly QUAD   ← 2 * ARITY
:EndNamespace

:Namespace RANGES
    :Access Public Shared
    :Field Public Shared ReadOnly BYTE  ← ⍳CONSTANTS.BYTE
    :Field Public Shared ReadOnly ROW   ← ⍳CONSTANTS.ROW
    :Field Public Shared ReadOnly PLANE ← ⍳CONSTANTS.PLANE
    :Field Public Shared ReadOnly CUBE  ← ⍳CONSTANTS.CUBE
:EndNamespace
```

Listing 12: `Iscandar.apl`

```
:Namespace Iscandar

    A←32 ⋄ OK←1

    ∇ Pointer ← Allocate Total;S;B
      ⍝ 1. Search for a starting point that fits the total request
      (S B) ← ##.Decision.Resolve Total

      ⍝ 2. If the search fails, return a record marked as invalid
      :If S = 0
          Pointer ← ⎕NS '' ⋄ Pointer.status ← 0
          →0
      :EndIf

      ⍝ 3. Create a record with the location and size
      Pointer ← ⎕NS ''
      Pointer.(base len status) ← B Total OK

      ⍝ 4. Mark the selected sectors as occupied in the grid
      Mark Pointer

      ⍝ 5. Return the record to the caller
    ∇

    ∇ S ← Free Pointer
      ⍝ 1. Check if the pointer is valid before proceeding
      :If 0 = Pointer.status ⋄ S ← 0 ⋄ →0 ⋄ :EndIf

      ⍝ 2. Clear the sectors in the memory grid
      Unmark Pointer

      ⍝ 3. Reset the pointer status to show it is no longer active
      Pointer.status ← 0
      S ← OK
    ∇

    ∇ Main
      ⍝ 1. Prepare physical lattice and state maps
      Initialize_Lattice

      ⍝ 2. Set environment constants and pathing
      'Iscandar Engine Online. System initialized.'
    ∇

    ∇ Cleanup
      ⍝ 1. Reset all hierarchical occupancy counters
      { ⍵.sectors ← 0 ⋄ ⍵.count ← 0 }¨ ##.Cube.Rows

      ⍝ 2. Release oracle handles and clear transient state
      'Lattice returned to void. Environment clear.'
    ∇

:EndNamespace
```

Listing 13: `WrappedPointer.apl`

```
:Namespace WrappedPointer
    ⍝ --- Iscandar Bijective Pointer Implementation ---
    ⍝ Layout: [V:1][R:1][S:15][L:15]
```

```
    (V R S L) ← 0 0 0 0

    ∇ Status ← Unwrap P;bits
      ⍝ 1. Extract Length (Bits 0-14)
      L ← 32767 ∧ P

      ⍝ 2. Extract Start Index S (Bits 15-29)
      ⍝ Equivalent to (P bitshift-right 15) AND 32767
      S ← 32767 ∧ P ⌊ 2*15

      ⍝ 3. Extract Control Bits
      R ← 1 ∧ P ⌊ 2*30      ⍝ Reserved
      V ← 1 ∧ P ⌊ 2*31      ⍝ Virtual/Real Flag

      Status ← 1 ⍝ STATUS_SUCCESS
    ∇

    ∇ Coord ← GetCoordinate
      ⍝ Implementation of Axiom V (Equation 5)
      ⍝ Decodes the scalar projection S into vector (p,r,s)
      Coord ← 32 32 32 ⊤ S
    ∇

    ∇ P ← Wrap(p r s len v);S_field
      ⍝ 1. Construct Start Index S from coordinate vector
      S_field ← 32 ⊥ p r s

      ⍝ 2. Pack into 32-bit Word: (V * 2^31) + (S * 2^15) + L
      P ← (v × 2*31) + (S_field × 2*15) + len
    ∇
:EndNamespace
```

Listing 14: `Cube.apl`

```
:Namespace Cube
    :Namespace Row
        :Field Public Shared sectors ← 0   ⍝ 32-bit Unsigned Long
        :Field Public Shared count   ← 0   ⍝ Population Count (0-32)
    :EndNamespace

    :Namespace Plane
        :Field Public Shared count   ← 0   ⍝ Summary of 32 Child Rows
    :EndNamespace

    ⍝ The Physical Lattice Arrays
    :Field Public Shared Rows   ← CONSTANTS.PLANE ⍴ ⊂Row
    :Field Public Shared Planes ← CONSTANTS.ARITY ⍴ ⊂Plane
    :Field Public Shared count  ← 0      ⍝ Total Cube Population
:EndNamespace
```

Listing 15: `Decision.apl`

```
:Namespace Decision
    (⎕IO ⎕ML) ← 0 1

    ⍝ --- Tier Mapping via Byte-Arity ---
    ⍝ B: 8, R: 32, P: 1024, C: 32768

    Table ← C ⍴ ⊂##.Heuristic.Null

    Table[1+⍳B] ← ⊂##.Heuristic.Nano
    Table[B+⍳(R-B)] ← ⊂##.Heuristic.Tiny
    Table[R+⍳R] ← ⊂##.Heuristic.Small
    Table[(2×R)+⍳(P-(2×R))] ← ⊂##.Heuristic.Large
    Table[P+⍳(C-P)] ← ⊂##.Heuristic.Huge
    Table[C] ← ⊂##.Heuristic.Cube

    ∇ R ← Resolve s;Tier
      :Access Public Shared

      ⍝ 1. Range Guard
      :If (s < 1) ∨ (s > C)
          R ← ##.STATUS_FAILURE 0 ⋄ :Return
      :EndIf

      ⍝ 2. Select the Namespace Reference
      Tier ← ⊃Table[s]

      ⍝ 3. Invoke standardized Dispatch
      R ← Tier.Dispatch s
    ∇
:EndNamespace
```

Listing 16: HeuristicCube.apl

```
:Namespace Heuristic
    :Namespace Cube
        (⎕IO ⎕ML) ← 0 1

        ∇ R ← Dispatch s
          :Access Public Shared
          ⍝ Tier-5: Maximum volume (32768 sectors)
          ⍝ 1. Check if the entire Cube is clear
          :If 32768 = ##.##.Lattice.TotalCount
              R ← ##.##.STATUS_SUCCESS ##.##.CubePointer
          :Else
              R ← ##.##.STATUS_FAILURE 0
          :EndIf
        ∇
    :EndNamespace
:EndNamespace
```

Listing 17: HeuristicNano.apl

```
:Namespace Heuristic
  :Namespace HeuristicNano
    ⍝ --- Constants ---
    OK ← CONSTANT.STATUS_SUCCESS
    FAIL ← CONSTANT.STATUS_FAILURE
    LIMIT ← 1023

    ⍝ --- Function: SectorChecker ---
    ⍝ Args: (Row Offset L)
    ∇ Result ← SectorChecker Args;Row;Offset;L
      ∇ Result ← SectorChecker Args;GlobalRow;X;S;P
      (Row Offset L) ← Args
      GlobalRow ← Row + Offset

      ⍝ Boundary Check
      →(GlobalRow > LIMIT)/FAIL_EXIT

      X ← Cube.Rows[GlobalRow]
      S ← X.sectors

      ⍝ 2D Indexing: Use Oracle as bit scan
      P ← Oracle.Center[S;L]

      →(P < 0)/FAIL_EXIT
      Result ← OK P ⋄ →END

      FAIL_EXIT:
        Result ← FAIL 0
      END:
    ∇

    ⍝ --- Function: Main ---
    ∇ Result ← Main L;Offset
      Offset ← 0
      Result ← SectorChecker RowsDriver.Run
                PlanesDriver.Run (Offset L)
    ∇
  :EndNamespace
:EndNamespace
```

Listing 18: HeuristicTiny.apl

```
:Namespace Heuristic
  :Namespace HeuristicTiny
    ⍝ --- Constants ---
    OK ← CONSTANT.STATUS_SUCCESS
    FAIL ← CONSTANT.STATUS_FAILURE
    LIMIT ← 1023

    ⍝ --- Function: SectorChecker ---
    ⍝ Args: (Row Offset L)
    ∇ Result ← SectorChecker Args;Row;Offset;L
      ∇ Result ← SectorChecker Args;GlobalRow;X;S;P
      (Row Offset L) ← Args
      GlobalRow ← Row + Offset

      ⍝ Boundary Check
      →(GlobalRow > LIMIT)/FAIL_EXIT

      X ← Cube.Rows[GlobalRow]
      S ← X.sectors

      ⍝ 2D Indexing: Use Oracle as bit scan
```

```
      P ← Oracle.Left[S;L]
      →(P < 0)/SECOND_CHANCE
      Result ← OK P ⋄ →END

      SECOND_CHANCE:
      P ← Oracle.Right[S;L]
      →(P < 0)/FAIL_EXIT
      Result ← OK P ⋄ →END

      FAIL_EXIT:
        Result ← FAIL 0
      END:
    ∇

    ⍝ --- Function: Main ---
    ∇ Result ← Main L;Offset
      Offset ← 0
      Result ← SectorChecker RowsDriver.Run
                PlanesDriver.Run (Offset L)
    ∇
  :EndNamespace
:EndNamespace
```

Listing 19: HeuristicSmall.apl

```
:Namespace Heuristic
  :Namespace HeuristicSmall
    ⍝ --- Constants ---
    OK ← CONSTANT.STATUS_SUCCESS
    FAIL ← CONSTANT.STATUS_FAILURE
    LIMIT ← 1022 ⍝ Pivot limit for pair lookahead

    ⍝ --- Function: SectorChecker ---
    ⍝ Args: (Row Offset L)
    ∇ Result ← SectorChecker Args;Row;Offset;L;GR1;GR2;S1;S2;P;Q;M
      (Row Offset L) ← Args
      GR1 ← Row + Offset
      GR2 ← GR1 + 1

      ⍝ Boundary Check: Final pivot is 1022
      →(GR1 > LIMIT)/FAIL_EXIT

      ⍝ Load Sectors from Cube
      S1 ← (Cube.Rows[GR1]).sectors
      S2 ← (Cube.Rows[GR2]).sectors

      ⍝ --- Scan Phase: S1 Left ---
      P ← Oracle.Left[S1]
      →(L > P)/SUPPLEMENT
      Result ← OK L ⋄ →END

    SUPPLEMENT:
      Q ← Oracle.Right[S2]
      M ← L - P
      →(M > Q)/FAIL_EXIT
      Result ← OK L ⋄ →END

    FAIL_EXIT:
        Result ← FAIL 0
      END:
    ∇

    ⍝ --- Function: Main ---
    ∇ Result ← Main L;Offset
      Offset ← 0
      Result ← SectorChecker PairsRowsDriver.Run
                PlanesDriver.Run (Offset L)
    ∇
  :EndNamespace
:EndNamespace
```

Listing 20: HeuristicLarge.apl

```
:Namespace HeuristicLarge

    ARITY ← 32 ⋄ OK ← 1 ⋄ FAIL ← 0

    ∇ Result ← Run L
      ⍝ Hierarchical Coordinate Dispatch
      Result ← ##.LargeRowDriver.Run (
          ##.LargeDriver.Run L SectorScan
      )
    ∇
```

```
    ∇ Result ← SectorScan (Plane Row L);W_Core;W_Edges;
                                       L_Edges;Idx;S;
                                       Sum;Start;Len
      ⍝ 1. Geometric Decomposition
      W_Core ← ⌊L ÷ ARITY
      W_Edges ← 2 × 0 ≠ ARITY | L
      Idx ← (Plane × 32) + Row

      ⍝ 2. Branch: Join vs. Extrusion
      :If W_Edges > 0
          L_Edges ← ARITY | L

          ⍝ Head Resolution
          S ← (##.Cube.Rows[Idx]).sectors
          (Len Start) ← ##.Oracle.Left.Resolve S
          L_Edges -← Len

          ⍝ Tail Resolution (Double-Ended Fast-Fail)
          :If L_Edges > 0
              S ← (##.Cube.Rows[Idx + 1 + W_Core]).sectors
              L_Edges -← ##.Oracle.Right.Resolve S
          :EndIf
          →(L_Edges > 0)/FAIL_EXIT

          ⍝ Shard (Core) Validation
          :If W_Core > 0
              Sum ← +/ (##.Cube.Rows[Idx + 1 + ⍳W_Core]).count
              →(Sum ≠ 0)/FAIL_EXIT
          :EndIf
          Result ← OK Start

      :Else
          ⍝ Case II: Row-Aligned Extrusion
          Sum ← +/ (##.Cube.Rows[Idx + ⍳W_Core]).count
          →(Sum ≠ 0)/FAIL_EXIT
          Result ← OK 0
      :EndIf
      →END

    FAIL_EXIT: Result ← FAIL 0
    END:
    ∇

:EndNamespace
```

Listing 21: `HeuristicHuge.apl`

```
:Namespace HeuristicHuge

    ∇ (S B) ← Run W_Core W_Rem
      ⍝ 1. Intent Check: Direct Core or Fragmented Remainder?
      :If W_Rem = 0
          ⍝ Invoke the pure monolithic radar
          (S B) ← ##.HugePlanesCoreDriver.Run W_Core
      :Else
          ⍝ Invoke the dual-track overspill radar
          (S B) ← ##.HugePlanesRemainDriver.Run W_Core W_Rem
      :EndIf

      ⍝ 2. Status Pass-Through
      ⍝ Success (S) and Absolute Sector Position (B)
      ⍝ are bubbled up to the top-level allocator.
    ∇

:EndNamespace
```

Listing 22: `MarkPlanes.apl`

```
:Namespace WrappedPointer

  ∇ Status ← MarkPlanes Pointer;plane;row;sector;length
    ∇ start_offset;lead_avail;lead_sectors;leftover
    ∇ sandwich_count;Delta;NewCounts;target_plane

    ⍝ 1. Find starting position and length
    (plane row sector length) ← Unwrap Pointer

    start_offset ← (row × ROW) + sector
    lead_avail ← PLANE - start_offset
    Delta ← ROW ⍴ 0

    ⍝ 2. Calculate sectors for each plane
    lead_sectors ← length ⌊ lead_avail
```

```
    Delta[plane + 1] ← lead_sectors

    leftover ← length - lead_sectors
    target_plane ← plane + 1

    :If leftover ≥ PLANE
        sandwich_count ← ⌊ leftover ÷ PLANE
        Delta[target_plane + ⍳sandwich_count] ← PLANE
        leftover ← leftover - (sandwich_count × PLANE)
        target_plane +← sandwich_count
    :EndIf

    :If leftover > 0
        Delta[target_plane + 1] ← leftover
    :EndIf

    ⍝ 3. Check for overflow
    NewCounts ← ##.Cube.Planes.count + Delta

    :If ∨/ NewCounts > PLANE
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 4. Save new counts and return success
    ##.Cube.Planes.count ← NewCounts
    Status ← 'STATUS_MARK_SUCCESS'
  ∇

:EndNamespace
```

Listing 23: `UnMarkPlanes.apl`

```
:Namespace WrappedPointer

  ∇ Status ← UnMarkPlanes Pointer;plane;row;sector;length
    ∇ start_offset;lead_avail;lead_sectors;leftover
    ∇ sandwich_count;Delta;NewCounts;target_plane

    ⍝ 1. Find starting position and length
    (plane row sector length) ← Unwrap Pointer

    start_offset ← (row × ROW) + sector
    lead_avail ← PLANE - start_offset
    Delta ← ROW ⍴ 0

    ⍝ 2. Calculate sectors for each plane
    lead_sectors ← length ⌊ lead_avail
    Delta[plane + 1] ← lead_sectors

    leftover ← length - lead_sectors
    target_plane ← plane + 1

    :If leftover ≥ PLANE
        sandwich_count ← ⌊ leftover ÷ PLANE
        Delta[target_plane + ⍳sandwich_count] ← PLANE
        leftover ← leftover - (sandwich_count × PLANE)
        target_plane +← sandwich_count
    :EndIf

    :If leftover > 0
        Delta[target_plane + 1] ← leftover
    :EndIf

    ⍝ 3. Check for underflow
    NewCounts ← ##.Cube.Planes.count - Delta

    :If ∨/ NewCounts < 0
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 4. Save new counts and return success
    ##.Cube.Planes.count ← NewCounts
    Status ← 'STATUS_MARK_SUCCESS'
  ∇

:EndNamespace
```

Listing 24: MarkSingleRow.apl

```
:Namespace WrappedPointer

  ∇ Status ← MarkSingleRow(plane row sector length);Mask
    ∇ RowBits;OverlapCount;global_row

    ⍝ 1. Calculate the global row index (0-1023)
    global_row ← (plane × ROW) + row

    ⍝ 2. Generate the allocation mask
    Mask ← sector ⌽ ROW ↑ length ⍴ 1

    ⍝ 3. Fetch the target row bits
    RowBits ← ##.Cube.Rows[global_row].sectors

    ⍝ 4. Verify the intersection is zero
    OverlapCount ← +/ RowBits ∧ Mask
    :If OverlapCount > 0
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 5. Verify row count won't breach limit
    :If (##.Cube.Rows[global_row].count + length) > SECTORS
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 6. Commit via bitwise OR
    ##.Cube.Rows[global_row].sectors ← RowBits ∨ Mask

    ⍝ 7. Commit count via addition
    ##.Cube.Rows[global_row].count +← length
    Status ← 'STATUS_MARK_SUCCESS'
  ∇

:EndNamespace
```

Listing 25: UnMarkSingleRow.apl

```
:Namespace WrappedPointer

  ∇ Status ← UnMarkSingleRow(plane row sector length);Mask
    ∇ RowBits;OverlapCount;global_row

    ⍝ 1. Calculate the global row index (0-1023)
    global_row ← (plane × ROW) + row

    ⍝ 2. Generate the allocation mask
    Mask ← sector ⌽ ROW ↑ length ⍴ 1

    ⍝ 3. Fetch the target row bits
    RowBits ← ##.Cube.Rows[global_row].sectors

    ⍝ 4. Verify the mask matches allocated sectors
    ⍝    (Ensure we are only clearing bits that are actually set)
    OverlapCount ← +/ RowBits ∧ Mask
    :If OverlapCount ≠ length
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 5. Verify row count won't underflow below zero
    :If (##.Cube.Rows[global_row].count - length) < 0
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 6. Commit via bitwise AND with inverted mask
    ##.Cube.Rows[global_row].sectors ← RowBits ∧ ~Mask

    ⍝ 7. Commit count via subtraction
    ##.Cube.Rows[global_row].count -← length
    Status ← 'STATUS_MARK_SUCCESS'
  ∇

:EndNamespace
```

Listing 26: MarkRows.apl

```
:Namespace WrappedPointer

  ∇ Status ← MarkRows(plane row sector length);A;Z;L_A;L_Z;Sequence;RowMatrix
```

```
    ∇ OverlapCounts

    ⍝ 1. Route Scenario A if the allocation fits inside a single row
    :If (sector + length) ≤ SECTORS
        Status ← MarkSingleRow plane row sector length
        :Return
    :EndIf

    ⍝ 2. Calculate global boundary row indices (0-1023)
    A ← (plane × ROW) + row
    Z ← A + ⌊(sector + length - 1) ÷ SECTORS

    ⍝ 3. Compute fractional edge lengths for boundary rows
    L_A ← SECTORS - sector
    L_Z ← SECTORS | (sector + length)

    ⍝ 4. Initialize bulk sequence with intermediate rows
    Sequence ← (A + 1) + ⍳(Z - A - 1)

    ⍝ 5. Evaluate and process Head Row A
    :If sector = 0
        Sequence ← A, Sequence
    :Else
        Status ← MarkFirstRow A sector L_A
        :If Status ≡ 'STATUS_MARK_FAILURE' ⋄ :Return ⋄ :EndIf
    :EndIf

    ⍝ 6. Evaluate and process Tail Row Z
    :If L_Z = 0
        Sequence ← Sequence, Z
    :Else
        Status ← MarkFinalRow Z L_Z
        :If Status ≡ 'STATUS_MARK_FAILURE' ⋄ :Return ⋄ :EndIf
    :EndIf

    ⍝ 7. Vectorized Collision Check using Index Projection
    ⍝    Extracts an [⍴Sequence × SECTORS] boolean matrix
    RowMatrix ← ↑##.Cube.Rows[Sequence].sectors

    ⍝ Sum across rows (horizontal reduction) to check for set bits
    OverlapCounts ← +/ RowMatrix
    :If ∨/ OverlapCounts > 0
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 8. Vectorized Commit and Count Update
    ##.Cube.Rows[Sequence].sectors ← ⊂SECTORS ⍴ 1
    ##.Cube.Rows[Sequence].count ← SECTORS

    Status ← 'STATUS_MARK_SUCCESS'
  ∇

:EndNamespace
```

Listing 27: `UnMarkRows.apl`

```
:Namespace WrappedPointer

  ∇ Status ← UnMarkRows(plane row sector length);A;Z;L_A;L_Z;Sequence;RowMatrix
    ∇ OverlapCounts

    ⍝ 1. Route Scenario A if the allocation fits inside a single row
    :If (sector + length) ≤ SECTORS
        Status ← UnMarkSingleRow plane row sector length
        :Return
    :EndIf

    ⍝ 2. Calculate global boundary row indices (0-1023)
    A ← (plane × ROW) + row
    Z ← A + ⌊(sector + length - 1) ÷ SECTORS

    ⍝ 3. Compute fractional edge lengths for boundary rows
    L_A ← SECTORS - sector
    L_Z ← SECTORS | (sector + length)

    ⍝ 4. Initialize bulk sequence with intermediate rows
    Sequence ← (A + 1) + ⍳(Z - A - 1)

    ⍝ 5. Evaluate and process Head Row A
    :If sector = 0
        Sequence ← A, Sequence
    :Else
```

```
        Status ← UnMarkFirstRow A sector L_A
        :If Status ≡ 'STATUS_MARK_FAILURE' ⋄ :Return ⋄ :EndIf
    :EndIf

    ⍝ 6. Evaluate and process Tail Row Z
    :If L_Z = 0
        Sequence ← Sequence, Z
    :Else
        Status ← UnMarkFinalRow Z L_Z
        :If Status ≡ 'STATUS_MARK_FAILURE' ⋄ :Return ⋄ :EndIf
    :EndIf

    ⍝ 7. Vectorized Allocation Check using Index Projection
    ⍝    Extracts an [⍴Sequence × SECTORS] boolean matrix
    RowMatrix ← ↑##.Cube.Rows[Sequence].sectors

    ⍝ Sum across rows to ensure all targeted sectors are fully allocated
    OverlapCounts ← +/ RowMatrix
    :If ∨/ OverlapCounts ≠ SECTORS
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 8. Vectorized Commit and Count Update (Clear Bits)
    ##.Cube.Rows[Sequence].sectors ← ⊂SECTORS ⍴ 0
    ##.Cube.Rows[Sequence].count ← 0

    Status ← 'STATUS_MARK_SUCCESS'
  ∇

:EndNamespace
```

Listing 28: `MarkFirstRow`

```
:Namespace WrappedPointer

  ∇ Status ← MarkFirstRow(global_row sector length);Mask;RowBits
    ∇ OverlapCount

    ⍝ 1. Generate the trailing-edge allocation mask
    ⍝    (Shifts 'length' bits to align with the starting sector)
    Mask ← sector ⍲ SECTORS ↑ length ⍴ 1

    ⍝ 2. Fetch the target global row bits
    RowBits ← ##.Cube.Rows[global_row].sectors

    ⍝ 3. Verify the intersection is zero (Collision Detection)
    OverlapCount ← +/ RowBits ∧ Mask
    :If OverlapCount > 0
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 4. Verify row count won't breach limit
    :If (##.Cube.Rows[global_row].count + length) > SECTORS
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 5. Commit via bitwise OR
    ##.Cube.Rows[global_row].sectors ← RowBits ∨ Mask

    ⍝ 6. Commit count via addition
    ##.Cube.Rows[global_row].count +← length
    Status ← 'STATUS_MARK_SUCCESS'
  ∇

:EndNamespace
```

Listing 29: `UnMarkFirstRow.apl`

```
:Namespace WrappedPointer

  ∇ Status ← UnMarkFirstRow(global_row sector length);Mask;RowBits
    ∇ OverlapCount

    ⍝ 1. Generate the trailing-edge allocation mask
    ⍝    (Shifts 'length' bits to align with the starting sector)
    Mask ← sector ⍲ SECTORS ↑ length ⍴ 1

    ⍝ 2. Fetch the target global row bits
    RowBits ← ##.Cube.Rows[global_row].sectors
```

```
    ⍝ 3. Verify the mask matches allocated sectors
    ⍝    (Ensure we are only clearing bits that are actually set)
    OverlapCount ← +/ RowBits ∧ Mask
    :If OverlapCount ≠ length
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 4. Verify row count won't underflow below zero
    :If (##.Cube.Rows[global_row].count - length) < 0
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 5. Commit via bitwise AND with inverted mask
    ##.Cube.Rows[global_row].sectors ← RowBits ∧ ~Mask

    ⍝ 6. Commit count via subtraction
    ##.Cube.Rows[global_row].count -← length
    Status ← 'STATUS_MARK_SUCCESS'
  ∇

:EndNamespace
```

Listing 30: `MarkFinalRow.apl`

```
:Namespace WrappedPointer

  ∇ Status ← MarkFinalRow(global_row length);Mask;RowBits
    ∇ OverlapCount

    ⍝ 1. Generate the front-aligned allocation mask
    ⍝    (Claims bits from index 0 up to 'length')
    Mask ← SECTORS ↑ length ⍴ 1

    ⍝ 2. Fetch the target global row bits
    RowBits ← ##.Cube.Rows[global_row].sectors

    ⍝ 3. Verify the intersection is zero (Collision Detection)
    OverlapCount ← +/ RowBits ∧ Mask
    :If OverlapCount > 0
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 4. Verify row count won't breach limit
    :If (##.Cube.Rows[global_row].count + length) > SECTORS
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 5. Commit via bitwise OR
    ##.Cube.Rows[global_row].sectors ← RowBits ∨ Mask

    ⍝ 6. Commit count via addition
    ##.Cube.Rows[global_row].count +← length
    Status ← 'STATUS_MARK_SUCCESS'
  ∇

:EndNamespace
```

Listing 31: `UnMarkFinalRow.apl`

```
:Namespace WrappedPointer

  ∇ Status ← UnMarkFinalRow(global_row length);Mask;RowBits
    ∇ OverlapCount

    ⍝ 1. Generate the front-aligned allocation mask
    ⍝    (Claims bits from index 0 up to 'length')
    Mask ← SECTORS ↑ length ⍴ 1

    ⍝ 2. Fetch the target global row bits
    RowBits ← ##.Cube.Rows[global_row].sectors

    ⍝ 3. Verify the mask matches allocated sectors
    ⍝    (Ensure we are only clearing bits that are actually set)
    OverlapCount ← +/ RowBits ∧ Mask
    :If OverlapCount ≠ length
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf
```

```
    ⍝ 4. Verify row count won't underflow below zero
    :If (##.Cube.Rows[global_row].count - length) < 0
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 5. Commit via bitwise AND with inverted mask
    ##.Cube.Rows[global_row].sectors ← RowBits ∧ ~Mask

    ⍝ 6. Commit count via subtraction
    ##.Cube.Rows[global_row].count -← length
    Status ← 'STATUS_MARK_SUCCESS'
  ∇

:EndNamespace
```

Listing 32: `Mark.apl`

```
:Namespace WrappedPointer

  ∇ Status ← Mark pointer;plane;row;sector;length

    ⍝ 1. Unwrap the structural geometry from the pointer
    (plane row sector length) ← Unwrap pointer

    ⍝ 2. Verify global cube count won't breach structural limits
    :If (##.Cube.count + length) > CUBE
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 3. Run high-level plane volume tracking (The Radar Tier)
    Status ← ##.Cube MarkPlanes plane row sector length
    :If Status ≡ 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 4. Run low-level structural bit allocation (The Actuator Tier)
    Status ← MarkRows plane row sector length
    :If Status ≡ 'STATUS_MARK_FAILURE'
        ⍝ Note: In a transactional failure state, a rollback of
        ⍝ the planes may be required if MarkPlanes modifies state
        :Return
    :EndIf

    ⍝ 5. Update global structural count upon absolute success
    ##.Cube.count +← length
    Status ← 'STATUS_MARK_SUCCESS'
  ∇

:EndNamespace
```

Listing 33: `UnMark.apl`

```
:Namespace WrappedPointer

  ∇ Status ← UnMark pointer;plane;row;sector;length

    ⍝ 1. Unwrap the structural geometry from the pointer
    (plane row sector length) ← Unwrap pointer

    ⍝ 2. Verify global cube count won't underflow below zero
    :If (##.Cube.count - length) < 0
        Status ← 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 3. Run high-level plane volume tracking rollback
    Status ← ##.Cube UnMarkPlanes plane row sector length
    :If Status ≡ 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 4. Run low-level structural bit deallocation
    Status ← UnMarkRows plane row sector length
    :If Status ≡ 'STATUS_MARK_FAILURE'
        :Return
    :EndIf

    ⍝ 5. Update global structural count upon absolute success
    ##.Cube.count -← length
    Status ← 'STATUS_MARK_SUCCESS'
  ∇
```

```
:EndNamespace
```

Listing 34: `HeuristicNull.apl`

```
:Namespace Heuristic
    :Namespace Null
        (⎕IO ⎕ML) ← 0 1

        ∇ R ← Dispatch s
          :Access Public Shared
          ⍝ Ground state: all requests here fail
          ⍝ Returns (STATUS_FAILURE, NullPointer)
          R ← ##.##.STATUS_FAILURE 0
        ∇
    :EndNamespace
:EndNamespace
```

Listing 35: `PlanesDriver.apl`

```
:Namespace PlanesDriver

    ⍝ --- Constants ---
    P ← CONSTANT.PLANE
    A ← CONSTANT.ROW
    OK ← 1 ⋄ FAIL ← 0

    ⍝ --- Constant Arity Weight Vector (Powers of 2) ---
    B ← 2*⍳32

    ⍝ --- Function: Run ---
    ∇ Result ← CR CS Run (Offset L);V
      V ← ExecutionVector L
      →(0 = V)/NULL_SET
      Result ← CS CR V Offset ExecuteVector L
      →END
    NULL_SET:
      Result ← FAIL 0
    END:
    ∇

    ∇ V ← ExecutionVector L;Available;Mask
      ⍝ Parallel hardware poll of all 32 plane capacities
      Available ← P - Cube.planes.count
      Mask ← L ≤ Available
      V ← Mask +.× B[⍳⍴Mask]
    ∇

    ∇ Result ← plane Position Args;offset;sector
      (offset sector) ← Args
      Result ← (CONSTANT.PLANE×plane)+(CONSTANT.ARITY×offset)+sector
    ∇

    ∇ Result ← CS CR V Offset ExecuteVector L;s;b;Candidate
      ⍝ Factor out bitwise intersection (Candidate planes)
      Candidate ← V ∧ B

      P_0:  →(0 = 0⊃Candidate)/P_1
            (s b) ← CS CR (0 Offset L)
            →(s ≠ OK)/P_1
            Result ← OK (0 Position (Offset b)) ⋄ →END
      P_1:  →(0 = 1⊃Candidate)/P_2
            (s b) ← CS CR (1 Offset L)
            →(s ≠ OK)/P_2
            Result ← OK (1 Position (Offset b)) ⋄ →END
      P_2:  →(0 = 2⊃Candidate)/P_3
            (s b) ← CS CR (2 Offset L)
            →(s ≠ OK)/P_3
            Result ← OK (2 Position (Offset b)) ⋄ →END
      P_3:  →(0 = 3⊃Candidate)/P_4
            (s b) ← CS CR (3 Offset L)
            →(s ≠ OK)/P_4
            Result ← OK (3 Position (Offset b)) ⋄ →END
      P_4:  →(0 = 4⊃Candidate)/P_5
            (s b) ← CS CR (4 Offset L)
            →(s ≠ OK)/P_5
            Result ← OK (4 Position (Offset b)) ⋄ →END
      P_5:  →(0 = 5⊃Candidate)/P_6
            (s b) ← CS CR (5 Offset L)
            →(s ≠ OK)/P_6
            Result ← OK (5 Position (Offset b)) ⋄ →END
      P_6:  →(0 = 6⊃Candidate)/P_7
            (s b) ← CS CR (6 Offset L)
            →(s ≠ OK)/P_7
```

```
      Result ← OK (6 Position (Offset b)) ⋄ →END
P_7:  →(0 = 7⊃Candidate)/P_8
      (s b) ← CS CR (7 Offset L)
      →(s ≠ OK)/P_8
      Result ← OK (7 Position (Offset b)) ⋄ →END
P_8:  →(0 = 8⊃Candidate)/P_9
      (s b) ← CS CR (8 Offset L)
      →(s ≠ OK)/P_9
      Result ← OK (8 Position (Offset b)) ⋄ →END
P_9:  →(0 = 9⊃Candidate)/P_10
      (s b) ← CS CR (9 Offset L)
      →(s ≠ OK)/P_10
      Result ← OK (9 Position (Offset b)) ⋄ →END
P_10: →(0 = 10⊃Candidate)/P_11
      (s b) ← CS CR (10 Offset L)
      →(s ≠ OK)/P_11
      Result ← OK (10 Position (Offset b)) ⋄ →END
P_11: →(0 = 11⊃Candidate)/P_12
      (s b) ← CS CR (11 Offset L)
      →(s ≠ OK)/P_12
      Result ← OK (11 Position (Offset b)) ⋄ →END
P_12: →(0 = 12⊃Candidate)/P_13
      (s b) ← CS CR (12 Offset L)
      →(s ≠ OK)/P_13
      Result ← OK (12 Position (Offset b)) ⋄ →END
P_13: →(0 = 13⊃Candidate)/P_14
      (s b) ← CS CR (13 Offset L)
      →(s ≠ OK)/P_14
      Result ← OK (13 Position (Offset b)) ⋄ →END
P_14: →(0 = 14⊃Candidate)/P_15
      (s b) ← CS CR (14 Offset L)
      →(s ≠ OK)/P_15
      Result ← OK (14 Position (Offset b)) ⋄ →END
P_15: →(0 = 15⊃Candidate)/P_16
      (s b) ← CS CR (15 Offset L)
      →(s ≠ OK)/P_16
      Result ← OK (15 Position (Offset b)) ⋄ →END
P_16: →(0 = 16⊃Candidate)/P_17
      (s b) ← CS CR (16 Offset L)
      →(s ≠ OK)/P_17
      Result ← OK (16 Position (Offset b)) ⋄ →END
P_17: →(0 = 17⊃Candidate)/P_18
      (s b) ← CS CR (17 Offset L)
      →(s ≠ OK)/P_18
      Result ← OK (17 Position (Offset b)) ⋄ →END
P_18: →(0 = 18⊃Candidate)/P_19
      (s b) ← CS CR (18 Offset L)
      →(s ≠ OK)/P_19
      Result ← OK (18 Position (Offset b)) ⋄ →END
P_19: →(0 = 19⊃Candidate)/P_20
      (s b) ← CS CR (19 Offset L)
      →(s ≠ OK)/P_20
      Result ← OK (19 Position (Offset b)) ⋄ →END
P_20: →(0 = 20⊃Candidate)/P_21
      (s b) ← CS CR (20 Offset L)
      →(s ≠ OK)/P_21
      Result ← OK (20 Position (Offset b)) ⋄ →END
P_21: →(0 = 21⊃Candidate)/P_22
      (s b) ← CS CR (21 Offset L)
      →(s ≠ OK)/P_22
      Result ← OK (21 Position (Offset b)) ⋄ →END
P_22: →(0 = 22⊃Candidate)/P_23
      (s b) ← CS CR (22 Offset L)
      →(s ≠ OK)/P_23
      Result ← OK (22 Position (Offset b)) ⋄ →END
P_23: →(0 = 23⊃Candidate)/P_24
      (s b) ← CS CR (23 Offset L)
      →(s ≠ OK)/P_24
      Result ← OK (23 Position (Offset b)) ⋄ →END
P_24: →(0 = 24⊃Candidate)/P_25
      (s b) ← CS CR (24 Offset L)
      →(s ≠ OK)/P_25
      Result ← OK (24 Position (Offset b)) ⋄ →END
P_25: →(0 = 25⊃Candidate)/P_26
      (s b) ← CS CR (25 Offset L)
      →(s ≠ OK)/P_26
      Result ← OK (25 Position (Offset b)) ⋄ →END
P_26: →(0 = 26⊃Candidate)/P_27
      (s b) ← CS CR (26 Offset L)
      →(s ≠ OK)/P_27
      Result ← OK (26 Position (Offset b)) ⋄ →END
P_27: →(0 = 27⊃Candidate)/P_28
      (s b) ← CS CR (27 Offset L)
      →(s ≠ OK)/P_28
```

```
            Result ← OK (27 Position (Offset b)) ⋄ →END
      P_28: →(0 = 28⊃Candidate)/P_29
            (s b) ← CS CR (28 Offset L)
            →(s ≠ OK)/P_29
            Result ← OK (28 Position (Offset b)) ⋄ →END
      P_29: →(0 = 29⊃Candidate)/P_30
            (s b) ← CS CR (29 Offset L)
            →(s ≠ OK)/P_30
            Result ← OK (29 Position (Offset b)) ⋄ →END
      P_30: →(0 = 30⊃Candidate)/P_31
            (s b) ← CS CR (30 Offset L)
            →(s ≠ OK)/P_31
            Result ← OK (30 Position (Offset b)) ⋄ →END
      P_31: →(0 = 31⊃Candidate)/FAIL_EXIT
            (s b) ← CS CR (31 Offset L)
            →(s ≠ OK)/FAIL_EXIT
            Result ← OK (31 Position (Offset b)) ⋄ →END

      FAIL_EXIT: Result ← FAIL 0
      END:
    ∇
:EndNamespace
```

Listing 36: `RowsDriver.apl`

```
:Namespace RowsDriver

    ⍝ --- Constants ---
    A ← CONSTANT.ROW
    OK ← CONSTANT.STATUS_SUCCESS
    FAIL ← CONSTANT.STATUS_FAILURE
    MAX_ROW ← 1023

    ⍝ --- Constant Arity Weight Vector ---
    B ← 2*⍳32

    ⍝ --- Function: Run ---
    ⍝ Args: (P Offset L)
    ∇ Result ← CS Run Args;V;P;Offset;L;Indices;Rows
      (P Offset L) ← Args

      ⍝ Generate 32 indices clipped at the 1023 hardware boundary
      ⍝ Note: CS must handle (P×32)+Offset > 1023 by returning FAIL
      Indices ← MAX_ROW ⌊ (P × CONSTANT.ARITY) + Offset + ⍳32
      Rows ← Cube.rows[Indices]

      V ← ExecutionVector (P Offset L)
      →(0 = V)/NULL_SET
      Result ← CS V ExecuteVector (P Offset L)
      →END
    NULL_SET:
      Result ← FAIL 0
    END:
    ∇

    ∇ V ← ExecutionVector Args;P;Offset;L;Counts;Mask
      (P Offset L) ← Args
      Counts ← Rows.count
      Mask ← L ≤ Counts
      V ← Mask +.× B
    ∇

    ∇ Result ← CS V ExecuteVector Args;P;Offset;L;s;b;Candidate
      (P Offset L) ← Args
      Candidate ← V ∧ B

      P_0:  →(0 = 0⊃Candidate)/P_1
            (s b) ← CS (0 Offset L) ⋄ →(s ≠ OK)/P_1
            Result ← OK((A×0)+b) ⋄ →END
      P_1:  →(0 = 1⊃Candidate)/P_2
            (s b) ← CS (1 Offset L) ⋄ →(s ≠ OK)/P_2
            Result ← OK((A×1)+b) ⋄ →END
      P_2:  →(0 = 2⊃Candidate)/P_3
            (s b) ← CS (2 Offset L) ⋄ →(s ≠ OK)/P_3
            Result ← OK((A×2)+b) ⋄ →END
      P_3:  →(0 = 3⊃Candidate)/P_4
            (s b) ← CS (3 Offset L) ⋄ →(s ≠ OK)/P_4
            Result ← OK((A×3)+b) ⋄ →END
      P_4:  →(0 = 4⊃Candidate)/P_5
            (s b) ← CS (4 Offset L) ⋄ →(s ≠ OK)/P_5
            Result ← OK((A×4)+b) ⋄ →END
      P_5:  →(0 = 5⊃Candidate)/P_6
            (s b) ← CS (5 Offset L) ⋄ →(s ≠ OK)/P_6
```

```
      Result ← OK((A×5)+b) ⋄ →END
P_6:  →(0 = 6⊃Candidate)/P_7
      (s b) ← CS (6 Offset L) ⋄ →(s ≠ OK)/P_7
      Result ← OK((A×6)+b) ⋄ →END
P_7:  →(0 = 7⊃Candidate)/P_8
      (s b) ← CS (7 Offset L) ⋄ →(s ≠ OK)/P_8
      Result ← OK((A×7)+b) ⋄ →END
P_8:  →(0 = 8⊃Candidate)/P_9
      (s b) ← CS (8 Offset L) ⋄ →(s ≠ OK)/P_9
      Result ← OK((A×8)+b) ⋄ →END
P_9:  →(0 = 9⊃Candidate)/P_10
      (s b) ← CS (9 Offset L) ⋄ →(s ≠ OK)/P_10
      Result ← OK((A×9)+b) ⋄ →END
P_10: →(0 = 10⊃Candidate)/P_11
      (s b) ← CS (10 Offset L) ⋄ →(s ≠ OK)/P_11
      Result ← OK((A×10)+b) ⋄ →END
P_11: →(0 = 11⊃Candidate)/P_12
      (s b) ← CS (11 Offset L) ⋄ →(s ≠ OK)/P_12
      Result ← OK((A×11)+b) ⋄ →END
P_12: →(0 = 12⊃Candidate)/P_13
      (s b) ← CS (12 Offset L) ⋄ →(s ≠ OK)/P_13
      Result ← OK((A×12)+b) ⋄ →END
P_13: →(0 = 13⊃Candidate)/P_14
      (s b) ← CS (13 Offset L) ⋄ →(s ≠ OK)/P_14
      Result ← OK((A×13)+b) ⋄ →END
P_14: →(0 = 14⊃Candidate)/P_15
      (s b) ← CS (14 Offset L) ⋄ →(s ≠ OK)/P_15
      Result ← OK((A×14)+b) ⋄ →END
P_15: →(0 = 15⊃Candidate)/P_16
      (s b) ← CS (15 Offset L) ⋄ →(s ≠ OK)/P_16
      Result ← OK((A×15)+b) ⋄ →END
P_17: →(0 = 17⊃Candidate)/P_18
      (s b) ← CS (17 Offset L) ⋄ →(s ≠ OK)/P_18
      Result ← OK((A×17)+b) ⋄ →END
P_18: →(0 = 18⊃Candidate)/P_19
      (s b) ← CS (18 Offset L) ⋄ →(s ≠ OK)/P_19
      Result ← OK((A×18)+b) ⋄ →END
P_19: →(0 = 19⊃Candidate)/P_20
      (s b) ← CS (19 Offset L) ⋄ →(s ≠ OK)/P_20
      Result ← OK((A×19)+b) ⋄ →END
P_20: →(0 = 20⊃Candidate)/P_21
      (s b) ← CS (20 Offset L) ⋄ →(s ≠ OK)/P_21
      Result ← OK((A×20)+b) ⋄ →END
P_21: →(0 = 21⊃Candidate)/P_22
      (s b) ← CS (21 Offset L) ⋄ →(s ≠ OK)/P_22
      Result ← OK((A×21)+b) ⋄ →END
P_22: →(0 = 22⊃Candidate)/P_23
      (s b) ← CS (22 Offset L) ⋄ →(s ≠ OK)/P_23
      Result ← OK((A×22)+b) ⋄ →END
P_23: →(0 = 23⊃Candidate)/P_24
      (s b) ← CS (23 Offset L) ⋄ →(s ≠ OK)/P_24
      Result ← OK((A×23)+b) ⋄ →END
P_24: →(0 = 24⊃Candidate)/P_25
      (s b) ← CS (24 Offset L) ⋄ →(s ≠ OK)/P_25
      Result ← OK((A×24)+b) ⋄ →END
P_25: →(0 = 25⊃Candidate)/P_26
      (s b) ← CS (25 Offset L) ⋄ →(s ≠ OK)/P_26
      Result ← OK((A×25)+b) ⋄ →END
P_26: →(0 = 26⊃Candidate)/P_27
      (s b) ← CS (26 Offset L) ⋄ →(s ≠ OK)/P_27
      Result ← OK((A×26)+b) ⋄ →END
P_27: →(0 = 27⊃Candidate)/P_28
      (s b) ← CS (27 Offset L) ⋄ →(s ≠ OK)/P_28
      Result ← OK((A×27)+b) ⋄ →END
P_28: →(0 = 28⊃Candidate)/P_29
      (s b) ← CS (28 Offset L) ⋄ →(s ≠ OK)/P_29
      Result ← OK((A×28)+b) ⋄ →END
P_29: →(0 = 29⊃Candidate)/P_30
      (s b) ← CS (29 Offset L) ⋄ →(s ≠ OK)/P_30
      Result ← OK((A×29)+b) ⋄ →END
P_30: →(0 = 30⊃Candidate)/P_31
      (s b) ← CS (30 Offset L) ⋄ →(s ≠ OK)/P_31
      Result ← OK((A×30)+b) ⋄ →END
P_31: →(0 = 31⊃Candidate)/FAIL_EXIT
      (s b) ← CS (31 Offset L) ⋄ →(s ≠ OK)/FAIL_EXIT
      Result ← OK((A×31)+b) ⋄ →END

FAIL_EXIT: Result ← FAIL 0
END:
  ∇
:EndNamespace
```

Listing 37: `PairRowsDriver.apl`

```
:Namespace PairRowsDriver

    ⍝ --- Constants ---
    A ← 32 ⍝ Standard Row Width (FIM-32 Lattice)
    OK ← 1 ⋄ FAIL ← 0

    ⍝ --- Function: Run ---
    ⍝ Entry point for the Small Heuristic (33-64 sectors).
    ∇ Result ← CS Offset CR Run R;V
      V ← ExecutionVector Offset R
      →(0 = V) / NULL_SET
      Result ← CS CR V Offset ExecuteVector R
      → END
      NULL_SET: Result ← FAIL 0
      END:
    ∇

    ⍝ --- Function: ExecutionVector ---
    ⍝ Generates a 32-bit radar mask of potential row-pair bridges.
    ∇ V ← ExecutionVector Offset R;F;Counts
      F ← Offset ⋄ V ← 0
      Counts ← Cube.Rows.count

      ⍝ Bit-weighted volume checks: (R ≤ Row[i].count + Row[i+1].count)
      V +← (R ≤ Counts[F+0]  + Counts[F+1])  × 2*0
      V +← (R ≤ Counts[F+1]  + Counts[F+2])  × 2*1
      V +← (R ≤ Counts[F+2]  + Counts[F+3])  × 2*2
      V +← (R ≤ Counts[F+3]  + Counts[F+4])  × 2*3
      V +← (R ≤ Counts[F+4]  + Counts[F+5])  × 2*4
      V +← (R ≤ Counts[F+5]  + Counts[F+6])  × 2*5
      V +← (R ≤ Counts[F+6]  + Counts[F+7])  × 2*6
      V +← (R ≤ Counts[F+7]  + Counts[F+8])  × 2*7
      V +← (R ≤ Counts[F+8]  + Counts[F+9])  × 2*8
      V +← (R ≤ Counts[F+9]  + Counts[F+10]) × 2*9
      V +← (R ≤ Counts[F+10] + Counts[F+11]) × 2*10
      V +← (R ≤ Counts[F+11] + Counts[F+12]) × 2*11
      V +← (R ≤ Counts[F+12] + Counts[F+13]) × 2*12
      V +← (R ≤ Counts[F+13] + Counts[F+14]) × 2*13
      V +← (R ≤ Counts[F+14] + Counts[F+15]) × 2*14
      V +← (R ≤ Counts[F+15] + Counts[F+16]) × 2*15
      V +← (R ≤ Counts[F+16] + Counts[F+17]) × 2*16
      V +← (R ≤ Counts[F+17] + Counts[F+18]) × 2*17
      V +← (R ≤ Counts[F+18] + Counts[F+19]) × 2*18
      V +← (R ≤ Counts[F+19] + Counts[F+20]) × 2*19
      V +← (R ≤ Counts[F+20] + Counts[F+21]) × 2*20
      V +← (R ≤ Counts[F+21] + Counts[F+22]) × 2*21
      V +← (R ≤ Counts[F+22] + Counts[F+23]) × 2*22
      V +← (R ≤ Counts[F+23] + Counts[F+24]) × 2*23
      V +← (R ≤ Counts[F+24] + Counts[F+25]) × 2*24
      V +← (R ≤ Counts[F+25] + Counts[F+26]) × 2*25
      V +← (R ≤ Counts[F+26] + Counts[F+27]) × 2*26
      V +← (R ≤ Counts[F+27] + Counts[F+28]) × 2*27
      V +← (R ≤ Counts[F+28] + Counts[F+29]) × 2*28
      V +← (R ≤ Counts[F+29] + Counts[F+30]) × 2*29
      V +← (R ≤ Counts[F+30] + Counts[F+31]) × 2*30

      →(F + 32 ≥ ⍴Counts) / END_V
      V +← (R ≤ Counts[F+31] + Counts[F+32]) × 2*31
      END_V:
    ∇

    ⍝ --- Function: ExecuteVector ---
    ⍝ Unrolled jump-table for sequential chance recovery.
    ∇ Result ← CS CR V Offset ExecuteVector R;s;b
      P_0:  →(0 = V ∧ 2*0)  / P_1 ⋄ (s b) ← CS 0 Offset R ⋄ →(s ≠ OK) / P_1
            Result ← OK((A×0)+b) ⋄ →END
      P_1:  →(0 = V ∧ 2*1)  / P_2 ⋄ (s b) ← CS 1 Offset R ⋄ →(s ≠ OK) / P_2
            Result ← OK((A×1)+b) ⋄ →END
      P_2:  →(0 = V ∧ 2*2)  / P_3 ⋄ (s b) ← CS 2 Offset R ⋄ →(s ≠ OK) / P_3
            Result ← OK((A×2)+b) ⋄ →END
      P_3:  →(0 = V ∧ 2*3)  / P_4 ⋄ (s b) ← CS 3 Offset R ⋄ →(s ≠ OK) / P_4
            Result ← OK((A×3)+b) ⋄ →END
      P_4:  →(0 = V ∧ 2*4)  / P_5 ⋄ (s b) ← CS 4 Offset R ⋄ →(s ≠ OK) / P_5
            Result ← OK((A×4)+b) ⋄ →END
      P_5:  →(0 = V ∧ 2*5)  / P_6 ⋄ (s b) ← CS 5 Offset R ⋄ →(s ≠ OK) / P_6
            Result ← OK((A×5)+b) ⋄ →END
      P_6:  →(0 = V ∧ 2*6)  / P_7 ⋄ (s b) ← CS 6 Offset R ⋄ →(s ≠ OK) / P_7
            Result ← OK((A×6)+b) ⋄ →END
      P_7:  →(0 = V ∧ 2*7)  / P_8 ⋄ (s b) ← CS 7 Offset R ⋄ →(s ≠ OK) / P_8
            Result ← OK((A×7)+b) ⋄ →END
      P_8:  →(0 = V ∧ 2*8)  / P_9 ⋄ (s b) ← CS 8 Offset R ⋄ →(s ≠ OK) / P_9
            Result ← OK((A×8)+b) ⋄ →END
```

```
      P_9:  →(0 = V ∧ 2*9)  / P_10 ⋄ (s b) ← CS 9 Offset R ⋄ →(s ≠ OK) / P_10
            Result ← OK((A×9)+b) ⋄ →END
      P_10: →(0 = V ∧ 2*10) / P_11 ⋄ (s b) ← CS 10 Offset R ⋄ →(s ≠ OK) / P_11
            Result ← OK((A×10)+b) ⋄ →END
      P_11: →(0 = V ∧ 2*11) / P_12 ⋄ (s b) ← CS 11 Offset R ⋄ →(s ≠ OK) / P_12
            Result ← OK((A×11)+b) ⋄ →END
      P_12: →(0 = V ∧ 2*12) / P_13 ⋄ (s b) ← CS 12 Offset R ⋄ →(s ≠ OK) / P_13
            Result ← OK((A×12)+b) ⋄ →END
      P_13: →(0 = V ∧ 2*13) / P_14 ⋄ (s b) ← CS 13 Offset R ⋄ →(s ≠ OK) / P_14
            Result ← OK((A×13)+b) ⋄ →END
      P_14: →(0 = V ∧ 2*14) / P_15 ⋄ (s b) ← CS 14 Offset R ⋄ →(s ≠ OK) / P_15
            Result ← OK((A×14)+b) ⋄ →END
      P_15: →(0 = V ∧ 2*15) / P_16 ⋄ (s b) ← CS 15 Offset R ⋄ →(s ≠ OK) / P_16
            Result ← OK((A×15)+b) ⋄ →END
      P_16: →(0 = V ∧ 2*16) / P_17 ⋄ (s b) ← CS 16 Offset R ⋄ →(s ≠ OK) / P_17
            Result ← OK((A×16)+b) ⋄ →END
      P_17: →(0 = V ∧ 2*17) / P_18 ⋄ (s b) ← CS 17 Offset R ⋄ →(s ≠ OK) / P_18
            Result ← OK((A×17)+b) ⋄ →END
      P_18: →(0 = V ∧ 2*18) / P_19 ⋄ (s b) ← CS 18 Offset R ⋄ →(s ≠ OK) / P_19
            Result ← OK((A×18)+b) ⋄ →END
      P_19: →(0 = V ∧ 2*19) / P_20 ⋄ (s b) ← CS 19 Offset R ⋄ →(s ≠ OK) / P_20
            Result ← OK((A×19)+b) ⋄ →END
      P_20: →(0 = V ∧ 2*20) / P_21 ⋄ (s b) ← CS 20 Offset R ⋄ →(s ≠ OK) / P_21
            Result ← OK((A×20)+b) ⋄ →END
      P_21: →(0 = V ∧ 2*21) / P_22 ⋄ (s b) ← CS 21 Offset R ⋄ →(s ≠ OK) / P_22
            Result ← OK((A×21)+b) ⋄ →END
      P_22: →(0 = V ∧ 2*22) / P_23 ⋄ (s b) ← CS 22 Offset R ⋄ →(s ≠ OK) / P_23
            Result ← OK((A×22)+b) ⋄ →END
      P_23: →(0 = V ∧ 2*23) / P_24 ⋄ (s b) ← CS 23 Offset R ⋄ →(s ≠ OK) / P_24
            Result ← OK((A×23)+b) ⋄ →END
      P_24: →(0 = V ∧ 2*24) / P_25 ⋄ (s b) ← CS 24 Offset R ⋄ →(s ≠ OK) / P_25
            Result ← OK((A×24)+b) ⋄ →END
      P_25: →(0 = V ∧ 2*25) / P_26 ⋄ (s b) ← CS 25 Offset R ⋄ →(s ≠ OK) / P_26
            Result ← OK((A×25)+b) ⋄ →END
      P_26: →(0 = V ∧ 2*26) / P_27 ⋄ (s b) ← CS 26 Offset R ⋄ →(s ≠ OK) / P_27
            Result ← OK((A×26)+b) ⋄ →END
      P_27: →(0 = V ∧ 2*27) / P_28 ⋄ (s b) ← CS 27 Offset R ⋄ →(s ≠ OK) / P_28
            Result ← OK((A×27)+b) ⋄ →END
      P_28: →(0 = V ∧ 2*28) / P_29 ⋄ (s b) ← CS 28 Offset R ⋄ →(s ≠ OK) / P_29
            Result ← OK((A×28)+b) ⋄ →END
      P_29: →(0 = V ∧ 2*29) / P_30 ⋄ (s b) ← CS 29 Offset R ⋄ →(s ≠ OK) / P_30
            Result ← OK((A×29)+b) ⋄ →END
      P_30: →(0 = V ∧ 2*30) / P_31 ⋄ (s b) ← CS 30 Offset R ⋄ →(s ≠ OK) / P_31
            Result ← OK((A×30)+b) ⋄ →END
      P_31: →(0 = V ∧ 2*31) / FAIL_EXIT
            →(32 + Offset > 1023) / FAIL_EXIT
            (s b) ← CS 31 Offset R ⋄ →(s ≠ OK) / FAIL_EXIT
            Result ← OK((A×31)+b) ⋄ →END

      FAIL_EXIT: Result ← FAIL 0
      END:
    ∇
:EndNamespace
```

Listing 38: `LargeDriver.apl`

```
:Namespace LargeDriver

    A ← 32
    OK ← 1 ⋄ FAIL ← 0
    B ← 2*⍳32

    ∇ Result ← CR CS Run L;W;Matrix;Mask;Capability;Offset
      ⍝ 1. Calculate W-row window for tripartite weld
      W ← (⌊L÷A) + (2 × 0 ≠ A|L)

      ⍝ 2. Global sweep: poll all 1024 rows for occupancy
      ⍝ Pad with ARITY (A) to deter edge-overflow baskets
      Matrix ← 32 32 ⍴ 1024 ↑ W +/ (##.Cube.Rows.count, (W-1)⍴A)

      ⍝ 3. Generate 32-bit steering map (Capability Vector)
      ⍝ Identify planes where (Max Window - Occupancy) >= L
      Mask ← ∨/ L ≤ (W × A) - Matrix
      Capability ← Mask +.× B

      ⍝ 4. Admissibility Gate: execute fast-fail if sum is zero
      →(0 = Capability)/FAIL_EXIT

      ⍝ 5. Phase II Dispatch: descend to unrolled plane logic
      Offset ← 0
      Result ← CS CR Capability Offset Matrix ExecutePlaneDispatch L
      →END
```

```
    FAIL_EXIT: Result ← FAIL 0
    END:
    ∇

    ∇ Result ← CS CR V Offset Matrix ExecutePlaneDispatch L;s;b;Track;Candidate
      Candidate ← V ∧ B

      P0:  →(0 = 0⊃Candidate)/P1 ⋄ Track ← Matrix[0;]
           (s b) ← CS CR (0 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P1 ⋄ Result ← OK (0 Position (Offset b)) ⋄ →END

      P1:  →(0 = 1⊃Candidate)/P2 ⋄ Track ← Matrix[1;]
           (s b) ← CS CR (1 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P2 ⋄ Result ← OK (1 Position (Offset b)) ⋄ →END

      P2:  →(0 = 2⊃Candidate)/P3 ⋄ Track ← Matrix[2;]
           (s b) ← CS CR (2 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P3 ⋄ Result ← OK (2 Position (Offset b)) ⋄ →END

      P3:  →(0 = 3⊃Candidate)/P4 ⋄ Track ← Matrix[3;]
           (s b) ← CS CR (3 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P4 ⋄ Result ← OK (3 Position (Offset b)) ⋄ →END

      P4:  →(0 = 4⊃Candidate)/P5 ⋄ Track ← Matrix[4;]
           (s b) ← CS CR (4 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P5 ⋄ Result ← OK (4 Position (Offset b)) ⋄ →END

      P5:  →(0 = 5⊃Candidate)/P6 ⋄ Track ← Matrix[5;]
           (s b) ← CS CR (5 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P6 ⋄ Result ← OK (5 Position (Offset b)) ⋄ →END

      P6:  →(0 = 6⊃Candidate)/P7 ⋄ Track ← Matrix[6;]
           (s b) ← CS CR (6 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P7 ⋄ Result ← OK (6 Position (Offset b)) ⋄ →END

      P7:  →(0 = 7⊃Candidate)/P8 ⋄ Track ← Matrix[7;]
           (s b) ← CS CR (7 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P8 ⋄ Result ← OK (7 Position (Offset b)) ⋄ →END

      P8:  →(0 = 8⊃Candidate)/P9 ⋄ Track ← Matrix[8;]
           (s b) ← CS CR (8 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P9 ⋄ Result ← OK (8 Position (Offset b)) ⋄ →END

      P9:  →(0 = 9⊃Candidate)/P10 ⋄ Track ← Matrix[9;]
           (s b) ← CS CR (9 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P10 ⋄ Result ← OK (9 Position (Offset b)) ⋄ →END

      P10: →(0 = 10⊃Candidate)/P11 ⋄ Track ← Matrix[10;]
           (s b) ← CS CR (10 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P11 ⋄ Result ← OK (10 Position (Offset b)) ⋄ →END

      P11: →(0 = 11⊃Candidate)/P12 ⋄ Track ← Matrix[11;]
           (s b) ← CS CR (11 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P12 ⋄ Result ← OK (11 Position (Offset b)) ⋄ →END

      P12: →(0 = 12⊃Candidate)/P13 ⋄ Track ← Matrix[12;]
           (s b) ← CS CR (12 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P13 ⋄ Result ← OK (12 Position (Offset b)) ⋄ →END

      P13: →(0 = 13⊃Candidate)/P14 ⋄ Track ← Matrix[13;]
           (s b) ← CS CR (13 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P14 ⋄ Result ← OK (13 Position (Offset b)) ⋄ →END

      P14: →(0 = 14⊃Candidate)/P15 ⋄ Track ← Matrix[14;]
           (s b) ← CS CR (14 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P15 ⋄ Result ← OK (14 Position (Offset b)) ⋄ →END

      P15: →(0 = 15⊃Candidate)/P16 ⋄ Track ← Matrix[15;]
           (s b) ← CS CR (15 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P16 ⋄ Result ← OK (15 Position (Offset b)) ⋄ →END

      P16: →(0 = 16⊃Candidate)/P17 ⋄ Track ← Matrix[16;]
           (s b) ← CS CR (16 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P17 ⋄ Result ← OK (16 Position (Offset b)) ⋄ →END

      P17: →(0 = 17⊃Candidate)/P18 ⋄ Track ← Matrix[17;]
           (s b) ← CS CR (17 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P18 ⋄ Result ← OK (17 Position (Offset b)) ⋄ →END

      P18: →(0 = 18⊃Candidate)/P19 ⋄ Track ← Matrix[18;]
           (s b) ← CS CR (18 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P19 ⋄ Result ← OK (18 Position (Offset b)) ⋄ →END

      P19: →(0 = 19⊃Candidate)/P20 ⋄ Track ← Matrix[19;]
```

```
           (s b) ← CS CR (19 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P20 ⋄ Result ← OK (19 Position (Offset b)) ⋄ →END

      P20: →(0 = 20⊃Candidate)/P21 ⋄ Track ← Matrix[20;]
           (s b) ← CS CR (20 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P21 ⋄ Result ← OK (20 Position (Offset b)) ⋄ →END

      P21: →(0 = 21⊃Candidate)/P22 ⋄ Track ← Matrix[21;]
           (s b) ← CS CR (21 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P22 ⋄ Result ← OK (21 Position (Offset b)) ⋄ →END

      P22: →(0 = 22⊃Candidate)/P23 ⋄ Track ← Matrix[22;]
           (s b) ← CS CR (22 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P23 ⋄ Result ← OK (22 Position (Offset b)) ⋄ →END

      P23: →(0 = 23⊃Candidate)/P24 ⋄ Track ← Matrix[23;]
           (s b) ← CS CR (23 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P24 ⋄ Result ← OK (23 Position (Offset b)) ⋄ →END

      P24: →(0 = 24⊃Candidate)/P25 ⋄ Track ← Matrix[24;]
           (s b) ← CS CR (24 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P25 ⋄ Result ← OK (24 Position (Offset b)) ⋄ →END

      P25: →(0 = 25⊃Candidate)/P26 ⋄ Track ← Matrix[25;]
           (s b) ← CS CR (25 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P26 ⋄ Result ← OK (25 Position (Offset b)) ⋄ →END

      P26: →(0 = 26⊃Candidate)/P27 ⋄ Track ← Matrix[26;]
           (s b) ← CS CR (26 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P27 ⋄ Result ← OK (26 Position (Offset b)) ⋄ →END

      P27: →(0 = 27⊃Candidate)/P28 ⋄ Track ← Matrix[27;]
           (s b) ← CS CR (27 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P28 ⋄ Result ← OK (27 Position (Offset b)) ⋄ →END

      P28: →(0 = 28⊃Candidate)/P29 ⋄ Track ← Matrix[28;]
           (s b) ← CS CR (28 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P29 ⋄ Result ← OK (28 Position (Offset b)) ⋄ →END

      P29: →(0 = 29⊃Candidate)/P30 ⋄ Track ← Matrix[29;]
           (s b) ← CS CR (29 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P30 ⋄ Result ← OK (29 Position (Offset b)) ⋄ →END

      P30: →(0 = 30⊃Candidate)/P31 ⋄ Track ← Matrix[30;]
           (s b) ← CS CR (30 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/P31 ⋄ Result ← OK (30 Position (Offset b)) ⋄ →END

      P31: →(0 = 31⊃Candidate)/FAIL_EXIT ⋄ Track ← Matrix[31;]
           (s b) ← CS CR (31 Offset Track L) ##.RowsDriver.VerifyWeld
           →(s ≠ OK)/FAIL_EXIT ⋄ Result ← OK (31 Position (Offset b)) ⋄ →END

    FAIL_EXIT: Result ← FAIL 0
    END:
    ∇

    ∇ Result ← plane Position Args;offset;sector
      (offset sector) ← Args
      Result ← (32*2)×plane + (32×offset) + sector
    ∇

:EndNamespace
```

Listing 39: `LargeRowDriver.apl`

```
:Namespace LargeRowDriver

    A ← 32
    OK ← 1 ⋄ FAIL ← 0
    B ← 2*⍳32

    ∇ Result ← CS Run (Plane Offset Track L);V;Cap;Cand
      ⍝ 1. Map Railroad Track to local Capability Vector
      V ← L ≤ (A × (⌊L÷A) + (2 × 0 ≠ A|L)) - Track
      Cap ← V +.× B

      ⍝ 2. Fast-Fail if plane volume is insufficient
      →(0 = Cap)/FAIL_EXIT

      ⍝ 3. Unrolled Row Dispatch
      Result ← CS Cap Plane Offset L ExecuteRowDispatch
      →END

    FAIL_EXIT: Result ← FAIL 0
    END:
```

```
∇

∇ Result ← CS V Plane Offset L ExecuteRowDispatch;Cand;s;b
  Cand ← V ∧ B

  R0:  →(0 = 0⊃Cand)/R1
       (s b) ← CS (Plane 0 L)
       →(s ≠ OK)/R1 ⋄ Result ← OK (0 Position b) ⋄ →END

  R1:  →(0 = 1⊃Cand)/R2
       (s b) ← CS (Plane 1 L)
       →(s ≠ OK)/R2 ⋄ Result ← OK (1 Position b) ⋄ →END

  R2:  →(0 = 2⊃Cand)/R3
       (s b) ← CS (Plane 2 L)
       →(s ≠ OK)/R3 ⋄ Result ← OK (2 Position b) ⋄ →END

  R3:  →(0 = 3⊃Cand)/R4
       (s b) ← CS (Plane 3 L)
       →(s ≠ OK)/R4 ⋄ Result ← OK (3 Position b) ⋄ →END

  R4:  →(0 = 4⊃Cand)/R5
       (s b) ← CS (Plane 4 L)
       →(s ≠ OK)/R5 ⋄ Result ← OK (4 Position b) ⋄ →END

  R5:  →(0 = 5⊃Cand)/R6
       (s b) ← CS (Plane 5 L)
       →(s ≠ OK)/R6 ⋄ Result ← OK (5 Position b) ⋄ →END

  R6:  →(0 = 6⊃Cand)/R7
       (s b) ← CS (Plane 6 L)
       →(s ≠ OK)/R7 ⋄ Result ← OK (6 Position b) ⋄ →END

  R7:  →(0 = 7⊃Cand)/R8
       (s b) ← CS (Plane 7 L)
       →(s ≠ OK)/R8 ⋄ Result ← OK (7 Position b) ⋄ →END

  R8:  →(0 = 8⊃Cand)/R9
       (s b) ← CS (Plane 8 L)
       →(s ≠ OK)/R9 ⋄ Result ← OK (8 Position b) ⋄ →END

  R9:  →(0 = 9⊃Cand)/R10
       (s b) ← CS (Plane 9 L)
       →(s ≠ OK)/R10 ⋄ Result ← OK (9 Position b) ⋄ →END

  R10: →(0 = 10⊃Cand)/R11
       (s b) ← CS (10 Offset Track L)
       →(s ≠ OK)/R11 ⋄ Result ← OK (10 Position b) ⋄ →END

  R11: →(0 = 11⊃Cand)/R12
       (s b) ← CS (11 Offset Track L)
       →(s ≠ OK)/R12 ⋄ Result ← OK (11 Position b) ⋄ →END

  R12: →(0 = 12⊃Cand)/R13
       (s b) ← CS (12 Offset Track L)
       →(s ≠ OK)/R13 ⋄ Result ← OK (12 Position b) ⋄ →END

  R13: →(0 = 13⊃Cand)/R14
       (s b) ← CS (13 Offset Track L)
       →(s ≠ OK)/R14 ⋄ Result ← OK (13 Position b) ⋄ →END

  R14: →(0 = 14⊃Cand)/R15
       (s b) ← CS (14 Offset Track L)
       →(s ≠ OK)/R15 ⋄ Result ← OK (14 Position b) ⋄ →END

  R15: →(0 = 15⊃Cand)/R16
       (s b) ← CS (15 Offset Track L)
       →(s ≠ OK)/R16 ⋄ Result ← OK (15 Position b) ⋄ →END

  R16: →(0 = 16⊃Cand)/R17
       (s b) ← CS (16 Offset Track L)
       →(s ≠ OK)/R17 ⋄ Result ← OK (16 Position b) ⋄ →END

  R17: →(0 = 17⊃Cand)/R18
       (s b) ← CS (17 Offset Track L)
       →(s ≠ OK)/R18 ⋄ Result ← OK (17 Position b) ⋄ →END

  R18: →(0 = 18⊃Cand)/R19
       (s b) ← CS (18 Offset Track L)
       →(s ≠ OK)/R19 ⋄ Result ← OK (18 Position b) ⋄ →END

  R19: →(0 = 19⊃Cand)/R20
       (s b) ← CS (19 Offset Track L)
       →(s ≠ OK)/R20 ⋄ Result ← OK (19 Position b) ⋄ →END
```

```
      R20: →(0 = 20⊃Cand)/R21
           (s b) ← CS (20 Offset Track L)
           →(s ≠ OK)/R21 ⋄ Result ← OK (20 Position b) ⋄ →END

      R21: →(0 = 21⊃Cand)/R22
           (s b) ← CS (21 Offset Track L)
           →(s ≠ OK)/R22 ⋄ Result ← OK (21 Position b) ⋄ →END

      R22: →(0 = 22⊃Cand)/R23
           (s b) ← CS (22 Offset Track L)
           →(s ≠ OK)/R23 ⋄ Result ← OK (22 Position b) ⋄ →END

      R23: →(0 = 23⊃Cand)/R24
           (s b) ← CS (23 Offset Track L)
           →(s ≠ OK)/R24 ⋄ Result ← OK (23 Position b) ⋄ →END

      R24: →(0 = 24⊃Cand)/R25
           (s b) ← CS (24 Offset Track L)
           →(s ≠ OK)/R25 ⋄ Result ← OK (24 Position b) ⋄ →END

      R25: →(0 = 25⊃Cand)/R26
           (s b) ← CS (25 Offset Track L)
           →(s ≠ OK)/R26 ⋄ Result ← OK (25 Position b) ⋄ →END

      R26: →(0 = 26⊃Cand)/R27
           (s b) ← CS (26 Offset Track L)
           →(s ≠ OK)/R27 ⋄ Result ← OK (26 Position b) ⋄ →END

      R27: →(0 = 27⊃Cand)/R28
           (s b) ← CS (27 Offset Track L)
           →(s ≠ OK)/R28 ⋄ Result ← OK (27 Position b) ⋄ →END

      R28: →(0 = 28⊃Cand)/R29
           (s b) ← CS (28 Offset Track L)
           →(s ≠ OK)/R29 ⋄ Result ← OK (28 Position b) ⋄ →END

      R29: →(0 = 29⊃Cand)/R30
           (s b) ← CS (29 Offset Track L)
           →(s ≠ OK)/R30 ⋄ Result ← OK (29 Position b) ⋄ →END

      R30: →(0 = 30⊃Cand)/R31
           (s b) ← CS (30 Offset Track L)
           →(s ≠ OK)/R31 ⋄ Result ← OK (30 Position b) ⋄ →END

      R31: →(0 = 31⊃Cand)/FAIL_EXIT
           (s b) ← CS (31 Offset Track L)
           →(s ≠ OK)/FAIL_EXIT ⋄ Result ← OK (31 Position b) ⋄ →END

    FAIL_EXIT: Result ← FAIL 0
    END:
    ∇

    ∇ Result ← p Position b
      Result ← (32*2)×p + b
    ∇

:EndNamespace
```

Listing 40: HugePlanesCoreDriver.apl

```
:Namespace HugePlanesCoreDriver

    A←32 ⋄ P_SIZE←1024 ⋄ C_SIZE←32 ⋄ OK←1

    ∇ (S B) ← Run W_Core;I;R;Σ;T;C;L_T
      ⍝ --- 1. Radar Generation ---

      ⍝ Create index map with a W_Core lookahead buffer
      I ← ⍳(P_SIZE + W_Core)

      ⍝ Ingest counts; inject 'A' (32) beyond P_SIZE to form Silicon Wall
      R ← { (⍵ > P_SIZE): A ⋄ ##.Cube.Rows[⍵].count }¨ I

      ⍝ Windowed sum: Occupancy total for every potential W_Core span
      Σ ← P_SIZE ↑ (W_Core +/ R)

      ⍝ Global Track: 1,024-bit rail marking valid monolithic voids
      T ← (Σ = 0)

      ⍝ Capability Vector: 32-bit summary (High if Plane contains a 1)
      C ← { ∨/ (T[(A × ⍵-1) + ⍳A]) }¨ ⍳C_SIZE

      ⍝ --- 2. Fully Unrolled Execution Manifold ---
```

```
P_0:  →(0 = 0⊃C)/P_1  ⋄ L_T ← T[⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T 0
      →(S = OK)/EXIT
P_1:  →(0 = 1⊃C)/P_2  ⋄ L_T ← T[A + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T A
      →(S = OK)/EXIT
P_2:  →(0 = 2⊃C)/P_3  ⋄ L_T ← T[(2×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (2×A)
      →(S = OK)/EXIT
P_3:  →(0 = 3⊃C)/P_4  ⋄ L_T ← T[(3×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (3×A)
      →(S = OK)/EXIT
P_4:  →(0 = 4⊃C)/P_5  ⋄ L_T ← T[(4×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (4×A)
      →(S = OK)/EXIT
P_5:  →(0 = 5⊃C)/P_6  ⋄ L_T ← T[(5×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (5×A)
      →(S = OK)/EXIT
P_6:  →(0 = 6⊃C)/P_7  ⋄ L_T ← T[(6×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (6×A)
      →(S = OK)/EXIT
P_7:  →(0 = 7⊃C)/P_8  ⋄ L_T ← T[(7×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (7×A)
      →(S = OK)/EXIT
P_8:  →(0 = 8⊃C)/P_9  ⋄ L_T ← T[(8×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (8×A)
      →(S = OK)/EXIT
P_9:  →(0 = 9⊃C)/P_10 ⋄ L_T ← T[(9×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (9×A)
      →(S = OK)/EXIT
P_10: →(0 = 10⊃C)/P_11 ⋄ L_T ← T[(10×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (10×A)
      →(S = OK)/EXIT
P_11: →(0 = 11⊃C)/P_12 ⋄ L_T ← T[(11×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (11×A)
      →(S = OK)/EXIT
P_12: →(0 = 12⊃C)/P_13 ⋄ L_T ← T[(12×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (12×A)
      →(S = OK)/EXIT
P_13: →(0 = 13⊃C)/P_14 ⋄ L_T ← T[(13×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (13×A)
      →(S = OK)/EXIT
P_14: →(0 = 14⊃C)/P_15 ⋄ L_T ← T[(14×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (14×A)
      →(S = OK)/EXIT
P_15: →(0 = 15⊃C)/P_16 ⋄ L_T ← T[(15×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (15×A)
      →(S = OK)/EXIT
P_16: →(0 = 16⊃C)/P_17 ⋄ L_T ← T[(16×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (16×A)
      →(S = OK)/EXIT
P_17: →(0 = 17⊃C)/P_18 ⋄ L_T ← T[(17×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (17×A)
      →(S = OK)/EXIT
P_18: →(0 = 18⊃C)/P_19 ⋄ L_T ← T[(18×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (18×A)
      →(S = OK)/EXIT
P_19: →(0 = 19⊃C)/P_20 ⋄ L_T ← T[(19×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (19×A)
      →(S = OK)/EXIT
P_20: →(0 = 20⊃C)/P_21 ⋄ L_T ← T[(20×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (20×A)
      →(S = OK)/EXIT
P_21: →(0 = 21⊃C)/P_22 ⋄ L_T ← T[(21×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (21×A)
      →(S = OK)/EXIT
P_22: →(0 = 22⊃C)/P_23 ⋄ L_T ← T[(22×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (22×A)
      →(S = OK)/EXIT
P_23: →(0 = 23⊃C)/P_24 ⋄ L_T ← T[(23×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (23×A)
      →(S = OK)/EXIT
P_24: →(0 = 24⊃C)/P_25 ⋄ L_T ← T[(24×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (24×A)
      →(S = OK)/EXIT
P_25: →(0 = 25⊃C)/P_26 ⋄ L_T ← T[(25×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (25×A)
      →(S = OK)/EXIT
P_26: →(0 = 26⊃C)/P_27 ⋄ L_T ← T[(26×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (26×A)
      →(S = OK)/EXIT
P_27: →(0 = 27⊃C)/P_28 ⋄ L_T ← T[(27×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (27×A)
```

```
            →(S = OK)/EXIT
      P_28: →(0 = 28⊃C)/P_29 ⋄ L_T ← T[(28×A) + ⍳A]
            (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (28×A)
            →(S = OK)/EXIT
      P_29: →(0 = 29⊃C)/P_30 ⋄ L_T ← T[(29×A) + ⍳A]
            (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (29×A)
            →(S = OK)/EXIT
      P_30: →(0 = 30⊃C)/P_31 ⋄ L_T ← T[(30×A) + ⍳A]
            (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (30×A)
            →(S = OK)/EXIT
      P_31: →(0 = 31⊃C)/FAIL ⋄ L_T ← T[(31×A) + ⍳A]
            (S B) ← ##.HugeRowsCoreDriver.Run W_Core L_T (31×A)
            →(S = OK)/EXIT

      FAIL: S ← 0 ⋄ B ← 0 ⋄ →0
      EXIT:
    ∇

:EndNamespace
```

Listing 41: HugeRowsCoreDriver.apl

```
:Namespace HugeRowsCoreDriver

    A←32 ⋄ OK←1

    ∇ (S B) ← Run W_Core L_T Base;R_Idx
      ⍝ 1. Fast-Fail: Bulk Zero-Check (OR-reduction)
      ⍝ If the rail is dark, exit immediately with Fail
      :If 0 = ∨/L_T ⋄ S←0 ⋄ B←0 ⋄ →0 ⋄ :EndIf

      ⍝ 2. Hardware Decode: Resolve leading edge offset
      R_Idx ← L_T ⍳ 1

      ⍝ 3. Synthesis: Calculate absolute bit offset
      S ← OK
      B ← (Base + R_Idx) × A
    ∇

:EndNamespace
```

Listing 42: HugePlanesRemainDriver.apl

```
:Namespace HugePlanesRemainDriver

    A←32 ⋄ P_SIZE←1024 ⋄ C_SIZE←32 ⋄ OK←1

    ∇ (S B) ← Run W_Core W_Rem;I;R;S_T;E_T;Track;C;L_T
      ⍝ --- 1. Dual-Track Radar Generation ---

      ⍝ Lookahead indices for Head, Shard, and Tail rows
      I ← ⍳(P_SIZE + W_Core + 1)
      R ← { (⍵ ≥ P_SIZE): A ⋄ ##.Cube.Rows[⍵].count }¨ I

      ⍝ Track 1: Shard Rail (Monolithic Void)
      ⍝ Must be zero between the Head and the Tail
      S_T ← P_SIZE ↑ ((W_Core +/ 1 ↓ R) = 0)

      ⍝ Track 2: Edges Rail (Combined Capacity)
      ⍝ Checks if Space[Head] + Space[Tail] ≥ W_Rem
      E_T ← P_SIZE ↑ ((A − R) + (A − (W_Core + 1) ↓ R) ≥ W_Rem)

      ⍝ Global Track: Intersection of Void and Edge Space
      Track ← S_T ∧ E_T

      ⍝ Capability Vector: 32-bit Planar Filter
      C ← { ∨/ (Track[(A × ⍵) + ⍳A]) }¨ ⍳C_SIZE

      ⍝ --- 2. Fully Unrolled Execution Manifold ---
      P_0:  →(0 = 0⊃C)/P_1  ⋄ L_T ← Track[⍳A]
            (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T 0
            →(S = OK)/EXIT
      P_1:  →(0 = 1⊃C)/P_2  ⋄ L_T ← Track[A + ⍳A]
            (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T A
            →(S = OK)/EXIT
      P_2:  →(0 = 2⊃C)/P_3  ⋄ L_T ← Track[(2×A) + ⍳A]
            (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (2×A)
            →(S = OK)/EXIT
      P_3:  →(0 = 3⊃C)/P_4  ⋄ L_T ← Track[(3×A) + ⍳A]
            (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (3×A)
            →(S = OK)/EXIT
      P_4:  →(0 = 4⊃C)/P_5  ⋄ L_T ← Track[(4×A) + ⍳A]
            (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (4×A)
```

```
      →(S = OK)/EXIT
P_5:  →(0 = 5⊃C)/P_6  ⋄ L_T ← Track[(5×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (5×A)
      →(S = OK)/EXIT
P_6:  →(0 = 6⊃C)/P_7  ⋄ L_T ← Track[(6×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (6×A)
      →(S = OK)/EXIT
P_7:  →(0 = 7⊃C)/P_8  ⋄ L_T ← Track[(7×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (7×A)
      →(S = OK)/EXIT
P_8:  →(0 = 8⊃C)/P_9  ⋄ L_T ← Track[(8×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (8×A)
      →(S = OK)/EXIT
P_9:  →(0 = 9⊃C)/P_10 ⋄ L_T ← Track[(9×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (9×A)
      →(S = OK)/EXIT
P_10: →(0 = 10⊃C)/P_11 ⋄ L_T ← Track[(10×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (10×A)
      →(S = OK)/EXIT
P_11: →(0 = 11⊃C)/P_12 ⋄ L_T ← Track[(11×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (11×A)
      →(S = OK)/EXIT
P_12: →(0 = 12⊃C)/P_13 ⋄ L_T ← Track[(12×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (12×A)
      →(S = OK)/EXIT
P_13: →(0 = 13⊃C)/P_14 ⋄ L_T ← Track[(13×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (13×A)
      →(S = OK)/EXIT
P_14: →(0 = 14⊃C)/P_15 ⋄ L_T ← Track[(14×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (14×A)
      →(S = OK)/EXIT
P_15: →(0 = 15⊃C)/P_16 ⋄ L_T ← Track[(15×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (15×A)
      →(S = OK)/EXIT
P_16: →(0 = 16⊃C)/P_17 ⋄ L_T ← Track[(16×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (16×A)
      →(S = OK)/EXIT
P_17: →(0 = 17⊃C)/P_18 ⋄ L_T ← Track[(17×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (17×A)
      →(S = OK)/EXIT
P_18: →(0 = 18⊃C)/P_19 ⋄ L_T ← Track[(18×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (18×A)
      →(S = OK)/EXIT
P_19: →(0 = 19⊃C)/P_20 ⋄ L_T ← Track[(19×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (19×A)
      →(S = OK)/EXIT
P_20: →(0 = 20⊃C)/P_21 ⋄ L_T ← Track[(20×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (20×A)
      →(S = OK)/EXIT
P_21: →(0 = 21⊃C)/P_22 ⋄ L_T ← Track[(21×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (21×A)
      →(S = OK)/EXIT
P_22: →(0 = 22⊃C)/P_23 ⋄ L_T ← Track[(22×A) + ⍳A]
      (S B) ← ##.HugeRowsCoreDriver.Run W_Core W_Rem L_T (22×A)
      →(S = OK)/EXIT
P_23: →(0 = 23⊃C)/P_24 ⋄ L_T ← Track[(23×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (23×A)
      →(S = OK)/EXIT
P_24: →(0 = 24⊃C)/P_25 ⋄ L_T ← Track[(24×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (24×A)
      →(S = OK)/EXIT
P_25: →(0 = 25⊃C)/P_26 ⋄ L_T ← Track[(25×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (25×A)
      →(S = OK)/EXIT
P_26: →(0 = 26⊃C)/P_27 ⋄ L_T ← Track[(26×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (26×A)
      →(S = OK)/EXIT
P_27: →(0 = 27⊃C)/P_28 ⋄ L_T ← Track[(27×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (27×A)
      →(S = OK)/EXIT
P_28: →(0 = 28⊃C)/P_29 ⋄ L_T ← Track[(28×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (28×A)
      →(S = OK)/EXIT
P_29: →(0 = 29⊃C)/P_30 ⋄ L_T ← Track[(29×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (29×A)
      →(S = OK)/EXIT
P_30: →(0 = 30⊃C)/P_31 ⋄ L_T ← Track[(30×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (30×A)
      →(S = OK)/EXIT
P_31: →(0 = 31⊃C)/FAIL ⋄ L_T ← Track[(31×A) + ⍳A]
      (S B) ← ##.HugeRowsRemainDriver.Run W_Core W_Rem L_T (31×A)
      →(S = OK)/EXIT

FAIL: S ← 0 ⋄ B ← 0 ⋄ →0
```

```
      EXIT:
    ∇

:EndNamespace
```

Listing 43: HugeRowsRemainDriver.apl

```
:Namespace HugeRowsRemainDriver

    A←32 ⋄ OK←1

    ∇ (S B) ← Run W_Core W_Rem L_T Base;R_Idx;H_Idx;H_Row;P_H;T_Rem;T_Idx;T_Row
      ⍝ 1. Rail Gate: Fast-fail if the track is dark
      :If 0 = ∨/L_T ⋄ →FAIL ⋄ :EndIf

      ⍝ 2. Hardware Decode: Resolve leading edge row offset
      R_Idx ← L_T ⍳ 1

      ⍝ 3. Oracle Extraction: Fetch sector-position from Head
      H_Idx ← Base + R_Idx
      H_Row ← ##.Cube.Rows[H_Idx].sectors
      P_H ← ##.Oracles.Left[H_Row]

      ⍝ 4. Oracle Validity Gate: Catch negative or OOB sentinels
      :If (P_H < 0) ∨ (P_H ≥ A) ⋄ →FAIL ⋄ :EndIf

      ⍝ 5. Debt Calculation: Determine if overspill is required
      T_Rem ← W_Rem - (A - P_H)

      ⍝ 6. Early Success Shunt: If Head absorbed W_Rem, exit now
      :If T_Rem ≤ 0 ⋄ →DONE ⋄ :EndIf

      ⍝ 7. Tail Gate: Verify Tail row can host the spillover
      ⍝ Stride: Head [1] + Shard [W_Core] = Tail at +1
      T_Idx ← H_Idx + W_Core + 1
      T_Row ← ##.Cube.Rows[T_Idx].sectors

      ⍝ Tail check: Row must have enough leading-edge space
      :If T_Rem > (A - T_Row) ⋄ →FAIL ⋄ :EndIf

      DONE:
      ⍝ 8. Success Synthesis
      S ← OK
      B ← (H_Idx × A) + P_H
      →EXIT

      FAIL: S ← 0 ⋄ B ← 0
      EXIT:
    ∇

:EndNamespace
```